[DETAILS REQUIRED] — Name of the University / Institute

[DETAILS REQUIRED] — Department

LABORATORY PROJECT REPORT

StructRender: Engine-Grounded Real-Time Neural Rendering of a Game in Learned Painterly Styles

Submitted by

[DETAILS REQUIRED] — Student name(s)

Register number: [DETAILS REQUIRED]

Course: [DETAILS REQUIRED]

Under the guidance of [DETAILS REQUIRED] — Supervisor name and designation

October 2026

ABSTRACT

Generative world models can produce playable game footage from pixels, but they hold no authoritative game state, so geometry, items and object appearance can drift. **StructRender** takes the opposite approach: a deterministic game engine (Godot 4) keeps sole authority over physics, game logic and entity identity, and a neural network only decides how each frame *looks*. Every frame, the engine exports its own structural buffers (depth, normals, albedo, analytic motion vectors and an entity-identity map), and a compact image-to-image network renders them in a painted style.

Because no paired data exists for artistic styles, a Stable Diffusion 1.5 + depth-ControlNet teacher generates targets offline, accepted only if they pass four pre-registered alignment gates. The deployed renderer is a U-Net whose previous-frame features are warped by the engine's motion field and gated by a disocclusion mask; a 16-dimensional style vector applied through FiLM layers lets one model render two styles. It was trained on 24 recorded play sessions (23,760 frames at 512 × 288; 18/3/3 session split) and deployed through ONNX and TensorRT in a C++ GDExtension.

On the held-out test sessions, the two-style model reaches 35.33 dB PSNR (oil) and 34.79 dB (watercolour), LPIPS 0.0225 and 0.0243, and a sharpness ratio of 0.946 and 0.937 against a 0.90 gate. It flickers about half as much as its own teacher (0.54× and 0.48×). Live in the playable game it runs at 44.6 fps (FP16; 4.53 ms per network step on an RTX 5060 Laptop GPU), with zero fallbacks over 3,000 frames and graceful recovery from injected failures. Negative results are reported as measured: a windowed-transformer renderer never matched the U-Net at parameter parity, and a persistent per-entity appearance memory (EPAM) did not improve output on this data.

Keywords: neural rendering, game engines, image-to-image translation, diffusion teacher, temporal consistency, TensorRT, real-time inference.

TABLE OF CONTENTS

ABSTRACT..... i
LIST OF TABLES..... iv
LIST OF FIGURES..... v
LIST OF ABBREVIATIONS..... vi
CHAPTER 1: INTRODUCTION..... 1
1.1 Project Overview..... 1
1.2 Problem Statement..... 2
1.3 Objectives..... 3
1.4 Scope of Work and Social Contribution..... 4
CHAPTER 2: LITERATURE REVIEW..... 5
2.1 Background..... 5
2.1.1 Review of Related Work..... 6
2.2 Research Gaps..... 9
CHAPTER 3: PROPOSED METHODOLOGY..... 11
3.1 Architecture in a Nutshell..... 11
3.1.1 Dataset Description..... 12
3.1.2 Preprocessing..... 14
3.1.3 Encoder Layer (Feature-Extraction and Rendering Network)..... 16
3.2 Hardware Specifications..... 19
3.3 Software Specifications..... 20
CHAPTER 4: PERFORMANCE EVALUATION..... 21
4.1 Experimental Setup..... 21
4.2 Evaluation Metrics..... 22
4.3 Results and Analysis..... 24
4.3.1 Training Behaviour..... 24
4.3.2 Ablation Ladder on the Seed Scene..... 24
4.3.3 The Deployed Two-Style Model on the Game..... 27
4.3.4 Value of Frame History..... 29
4.3.5 Live Equivalence: the Game Shows the Evaluated Model..... 29
4.3.6 Real-Time Performance and Robustness..... 30
4.3.7 Assessment of the Objectives..... 32
4.3.8 Failure Cases and Limitations..... 32
CHAPTER 5: CONCLUSION AND FUTURE WORK..... 34

5.1 Summary of the Contributions....34
5.2 Future Work....35
REFERENCES....36
APPENDICES....39
Appendix A: Deployed Model Configuration (abridged)....39
Appendix B: Milestone Scorecard....39
Appendix C: Reproduction Commands and Evidence Files....39
Appendix D: Frames Excluded by the Alignment Gates....40

LIST OF TABLES

Table 2.1: Comparison of related work....8
Table 2.2: Research gaps and how StructRender addresses them....10
Table 3.1: Dataset summary....13
Table 3.2: Contents of one recorded frame and their use....13
Table 3.3: Offline teacher configuration....14
Table 3.4: Alignment gates (thresholds fixed before the data)....15
Table 3.5: Layer summary of StyleTemporalUNet (parameters counted from the model)….18
Table 3.6: Hardware used for all experiments....19
Table 3.7: Software environment (versions read from the environment)....20
Table 4.1: Training and evaluation configuration of the deployed model....21
Table 4.2: Ablation ladder: each rung adds one idea....22
Table 4.3: Ablation ladder, sealed test split (3,960 frames)....24
Table 4.4: Temporal error on 501 moving test pairs (lower is steadier)....25
Table 4.5: EPAM against its no-memory twin (sealed test, 3,912 clips, two styles)....27
Table 4.6: Game test sessions, 2,968 frames per style....27
Table 4.7: Mean L1 between each style's output and each style's targets (sampled)....28
Table 4.8: Style acquisition on the game test frames....28
Table 4.9: The same model with and without one frame of history (test frames)....29
Table 4.10: Equivalence chain from evaluator to live game....30
Table 4.11: M10 exit criteria, measured on the playable game....31
Table 4.12: Objectives and the evidence for each....32
Table B.1: Milestones M0–M10 and their verdicts....39
Table C.1: Evidence files behind the reported numbers....40
Table D.1: The 11 excluded frames (game dataset)....40

LIST OF FIGURES

Figure 1.1: The playable game rendered natively and by live StructRender (watercolour)……2
Figure 3.1: System architecture: offline data and supervision, and the runtime loop.....11
Figure 3.2: Structural inputs, motion and confidence mask for one test frame.....16
Figure 3.3: StyleTemporalUNet: style-free encoder, temporal fusion, FiLM decoder.....17
Figure 4.1: Training objective and validation L1/PSNR of the deployed model.....24
Figure 4.2: Ablation ladder: PSNR and sharpness on the sealed test split.....25
Figure 4.3: Temporal error of every rung against the engine and the teacher.....26
Figure 4.4: Flicker on the game's test sessions: engine, teacher and model.....27
Figure 4.5: Native input, teacher target and model output for one sealed test frame.....28
Figure 4.6: Image quality with and without frame history.....29
Figure 4.7: Live frame rate of the whole loop, and TensorRT latency alone.....30
Figure 4.8: Per-frame time over the 3,000-frame stability run.....31

LIST OF ABBREVIATIONS

AMP	Automatic mixed precision
CNN	Convolutional neural network
C++ GDExtension	Godot's native-code extension interface
EPAM	Engine-Keyed Persistent Appearance Memory
FiLM	Feature-wise linear modulation
FP16 / FP32 / TF32	16-bit, 32-bit and TensorFloat-32 floating-point formats
fps	Frames per second
GN	Group normalisation
GPU	Graphics processing unit
KB	The project's knowledge base and Decision Log
L1	Mean absolute error
LPIPS	Learned Perceptual Image Patch Similarity
ONNX	Open Neural Network Exchange
PSNR	Peak signal-to-noise ratio
RCS	Reappearance Consistency Score
RGB	Red–green–blue colour image
SD 1.5	Stable Diffusion version 1.5
SSIM	Structural similarity index
TE	Temporal (motion-warp) error
UV	Normalised screen coordinates in [0, 1]
VRAM	GPU video memory

CHAPTER 1: INTRODUCTION

1.1 Project Overview

Real-time rendering in games is built on a well-understood pipeline: the engine simulates the world, and a rasteriser turns the scene into pixels using hand-authored materials, lights and shaders. Changing the *look* of a game, for example rendering it as an oil painting or a watercolour, normally means re-authoring those assets and shaders. Neural rendering offers a different route: a network can learn to map scene information to an image in a target appearance [1]. Since deferred shading was introduced, engines have kept per-pixel scene attributes such as depth, normals and albedo in a so-called G-buffer [2], and these are exactly the kind of structured input such a network can consume.

A second line of work goes further and replaces the engine itself with a generative model. GameGAN [3], GameNGen [4], DIAMOND [5] and Genie [6] learn to predict the next frame of a game from previous frames and player actions. These systems are impressive, but they hold no authoritative state: whether a coin was collected, where a wall stands or which enemy is which is inferred from pixels, so it can drift over time. For a playable game this is a correctness problem, not only a visual one.

StructRender takes the opposite position. **The engine remains the single source of truth** for physics, game logic and entity identity; the neural renderer decides appearance only. Each frame the Godot engine [7] exports its structural buffers, and a compact network converts them into a styled RGB frame. Because the network never controls gameplay, a style can be switched in the middle of play without any change to the game state. Paired data for painterly styles does not exist, so an offline diffusion model (Stable Diffusion 1.5 [8] with a depth ControlNet [9]) is used only to generate training targets. The model that runs inside the game is a small convolutional network [10] exported through ONNX [11] to TensorRT [12] and driven from C++.

Figure 1.1 shows the outcome: the playable game rendered natively by Godot (left) and, for the same game state, by the live StructRender renderer (right). The project was organised as eleven milestones (M0–M10), each with exit criteria fixed in the project knowledge base before the work began [13]. Thresholds used to judge results were also written down before the results existed (pre-registration), and negative findings are reported rather than tuned away.

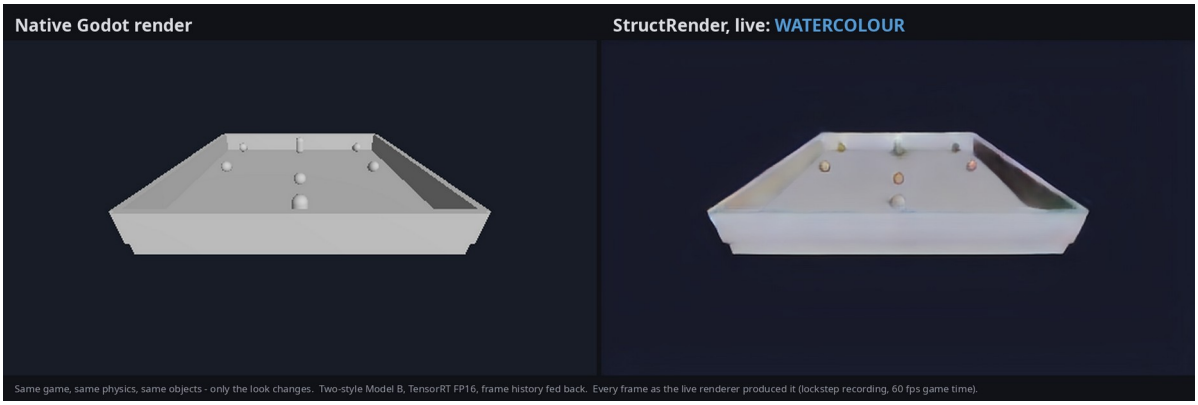


Figure 1.1: The playable game rendered natively and by live StructRender (watercolour)

The expected outcomes were a working end-to-end system (engine instrumentation, dataset, teacher, training, evaluation and live deployment), an honest ablation study of the architectural ideas the project set out to test, and a playable demonstration. Potential applications include stylised rendering for indie and educational games, rapid look-development for prototypes, and research on engine-grounded generative rendering.

1.2 Problem Statement

Problem. Given the deterministic structure a game engine already computes for each frame, render that frame in a learned artistic style at interactive frame rates, such that (i) the engine, not the network, determines what exists and where; (ii) the output is temporally stable; and (iii) the renderer can switch between styles while the game runs.

Shortcomings of existing approaches. Pixel-space world models do not own game state and can drift [4], [6]. Image-enhancement methods that consume G-buffers target photorealism and rely on large collections of real photographs as the style domain [14]. Per-frame image-to-image or style-transfer networks flicker when applied to video unless temporal consistency is enforced [15], [16]. Diffusion models produce high-quality stylisations but need many network evaluations per image [17], [8], which rules them out as the per-frame renderer on a laptop GPU.

Technical challenges. There is no paired ground truth for painted game frames. The teacher can invent geometry that is not in the scene. Teacher outputs are themselves temporally inconsistent. The live renderer must fit within a frame budget on an 8 GB laptop GPU, and its output in the game must be provably the same model that was evaluated offline.

Measurable outcome. On held-out play sessions: sharpness of at least 0.90 of the teacher's, colour-histogram travel of at least 0.80 of the teacher's, a temporal-error reduction larger than

a pre-registered noise threshold, about 30 fps for the complete live loop at 512 × 288, a standalone network latency of at most about 20 ms, and no gameplay interruption when the renderer fails.

Intended users. Game developers and researchers who want a learned visual style without giving up deterministic gameplay; and, for this report, evaluators assessing the technical contribution.

1.3 Objectives

Primary objective. To design, implement and evaluate an engine-grounded neural renderer that re-renders a playable Godot game in learned painterly styles in real time, with the engine retaining authority over all game state.

Secondary objectives.

1. **O1 – Structural data pipeline.** Export depth, normals, albedo, motion vectors, entity-ID maps, camera matrices and entity lifecycle events from Godot, and record validated, frame-aligned datasets.
2. **O2 – Offline supervision.** Generate styled training targets with a diffusion teacher, and accept only targets that pass four pre-defined alignment gates.
3. **O3 – Image quality.** Train a renderer whose held-out sharpness ratio is at least 0.90 and whose colour-histogram travel is at least 0.80 of the teacher's.
4. **O4 – Temporal stability.** Reduce temporal error through engine-guided reprojection by more than a pre-registered noise threshold.
5. **O5 – Controlled ablation.** Compare U-Net and transformer backbones, temporal conditioning, structured entity use and appearance memory at parameter parity, scored by one evaluation path on sealed test sessions.
6. **O6 – Live style switching.** Render two styles with one model and switch between them during play.
7. **O7 – Real-time deployment.** Run the model in the game through ONNX, TensorRT and a C++ GDExtension at about 30 fps (512 × 288), with standalone latency of at most about 20 ms and a native-frame fallback on failure.

8. **O8 – Persistent appearance memory.** Test whether EPAM, a per-entity appearance memory keyed by engine identity, improves output over an otherwise identical no-memory model.

1.4 Scope of Work and Social Contribution

Included. One Godot 4 game (an arena collector with a player, coins and spawned enemies); structural instrumentation; a dataset recorder and validator; an offline teacher with alignment gates; training and evaluation code for six renderer variants plus controls; an ONNX → TensorRT → C++ → Godot runtime; and a playable demonstration with live style switching. The development resolution is 512 × 288 and two styles are deployed: *oil impasto* and a *bold watercolour*.

Boundaries. The network never computes gameplay state. EPAM is session-local; there is no cross-session memory and no zero-shot recognition of unseen entities. Runtime diffusion, one-step diffusion transformers and mandatory adversarial training are explicitly out of scope [13]. Results are measured on one game and one recorded scene family; they are not evidence about other games or art styles. An attempt to move to 768 × 432 was stopped before the bulk run, because the teacher's look changes with resolution (Section 4.3.8).

Application environment. A single consumer laptop with an NVIDIA RTX 5060 Laptop GPU (8 GB) is used for development, training and deployment (Section 3.2).

Social and educational contribution. The work shows that a learned visual style can be added to a game without letting a generative model alter gameplay, a property that matters for fairness and correctness in interactive software. It runs on consumer hardware, and the whole pipeline, including its negative results and pre-registered thresholds, is documented, which makes it useful as a teaching example of evaluation discipline in applied deep learning. The impact is modest and should not be overstated: the styles are mild and the evidence covers one simple game.

Ethical, privacy and safety considerations. All data is synthetic and recorded from the project's own game; no personal data is collected. The teacher models are used under their published licences (CreativeML OpenRAIL-M for Stable Diffusion 1.5 [18]; Apache-2.0 for the ControlNet depth model [19]). Because the renderer cannot change game state, a renderer error cannot cause a gameplay error; on failure the native frame is shown.

CHAPTER 2: LITERATURE REVIEW

2.1 Background

G-buffers and deferred rendering. Saito and Takahashi introduced the idea of storing per-pixel geometric attributes in separate buffers and shading them in a later pass [2]. Modern engines, including Godot's Forward+ renderer [7], compute depth, normals and material properties every frame. StructRender re-renders these attributes with override shaders and exports them as the network's input, so the network is told the scene structure rather than asked to infer it.

Convolutional image-to-image networks. The U-Net [10] is an encoder–decoder CNN with skip connections that pass high-resolution encoder features directly to the decoder, preserving spatial detail. It became the standard generator for dense image translation, for example in pix2pix [20]. StructRender's renderer is a four-level U-Net with group normalisation [21], chosen because group normalisation does not depend on batch size and exports cleanly to ONNX.

Neural style transfer and conditioning. Gatys et al. showed that artistic style can be represented by correlations of deep CNN features [22]. Johnson et al. replaced the slow optimisation with a feed-forward network trained with perceptual losses [23]. Dumoulin et al. showed that one network can represent many styles through per-style normalisation parameters [24], and Huang and Belongie generalised this with adaptive instance normalisation [25]. FiLM [26] modulates features with a learned per-channel scale and shift computed from a conditioning vector. StructRender's two-style decoder uses FiLM driven by a learned 16-dimensional style vector.

Diffusion models and controllable editing. Denoising diffusion models generate images by iteratively removing noise [17]; latent diffusion runs this process in a compressed latent space, which made Stable Diffusion practical on consumer GPUs [8]. Image-to-image editing starts the reverse process from a partially noised input; the noise level (the *strength*) trades faithfulness against change [27]. ControlNet attaches a trainable copy of the encoder, connected through zero-initialised convolutions, so that a frozen diffusion model can be conditioned on depth, edges or pose [9]. StructRender conditions the teacher on the engine's exported depth.

Perceptual quality measures. SSIM compares local luminance, contrast and structure [28]. LPIPS measures distance between deep network features (here an AlexNet trunk [29]) and correlates well with human judgements [30]. Blau and Michaeli proved that distortion and perceptual quality are in tension [31], which explains why a regressor trained only on pixel error produces blurred output; the project therefore pairs pixel metrics with a sharpness measure.

Temporal reprojection. Real-time graphics reuses previous frames by warping them with motion vectors and rejecting history where it is invalid. Chaitanya et al. combined recurrence and auxiliary buffers for temporally stable denoising [32], and Xiao et al. used engine motion vectors to warp past frames for neural supersampling [33]. StructRender applies the same idea at the network's bottleneck, using analytic engine motion rather than estimated optical flow.

Window transformers. Swin Transformer restricts self-attention to local windows that shift between layers, giving linear cost in image size [34]. The project's *StructFormer* variant uses this kind of window attention behind a convolutional stem and was evaluated against the U-Net at equal parameter count.

2.1.1 Review of Related Work

The works below are grouped by the question they answer. For each, the problem, method, data, main finding and relevance to this project are summarised; Table 2.1 condenses the comparison. Bibliographic details were checked against the publishers' DOI records (Crossref) or the arXiv record.

Nalbach et al., "Deep Shading," Computer Graphics Forum, 2017 [35]. *Problem:* approximate expensive screen-space shading effects. *Method:* a CNN maps G-buffer attributes (positions, normals, material terms) to effects such as ambient occlusion and indirect light. *Data:* images rendered from 3D scenes. *Finding:* a CNN can learn screen-space shading from deferred buffers at interactive rates. *Limitation:* single frames, photographic shading goals, no learned artistic style. *Relevance:* the closest precedent for using engine buffers as network input.

Chaitanya et al., ACM TOG, 2017 [32]. *Problem:* reconstruct noisy low-sample Monte Carlo renders interactively. *Method:* a recurrent denoising autoencoder with auxiliary per-pixel features (normals, depth, roughness). *Data:* path-traced image sequences. *Finding:* recurrent connections improve temporal stability of the reconstruction. *Limitation:* reconstruction rather than restyling. *Relevance:* motivates feeding history back through the network.

Xiao et al., "Neural Supersampling for Real-time Rendering," ACM TOG, 2020 [33]. *Problem:* upsample low-resolution renders in real time. *Method:* warp several previous frames with engine motion vectors and depth, then reconstruct with a CNN. *Data:* rendered game-like scenes. *Finding:* engine motion makes multi-frame reconstruction practical in real time. *Limitation:* targets the engine's own appearance, not a new style. *Relevance:* supports using exact engine motion instead of estimated flow.

Isola et al., "pix2pix," CVPR 2017 [20] and **Park et al., "SPADE," CVPR 2019** [36]. *Problem:* translate label maps or other structure into images. *Method:* a conditional GAN with a U-Net generator and patch discriminator (pix2pix); spatially-adaptive normalisation driven by semantic layouts (SPADE). *Data:* paired datasets such as Cityscapes and facades (pix2pix), and COCO-Stuff, ADE20K and Cityscapes (SPADE). *Finding:* structure-to-image translation works when paired data exists. *Limitation:* needs paired data; no temporal model. *Relevance:* the image-translation framing StructRender adopts; StructRender replaces the missing pairs with a teacher.

Wang et al., "Video-to-Video Synthesis," NeurIPS 2018 [37] and **Mallya et al., "World-Consistent vid2vid," ECCV 2020** [38]. *Problem:* temporally coherent video from semantic inputs, and consistency when the camera revisits a place. *Method:* sequential generators with flow-based warping; the second work adds guidance images projected from a 3D point cloud of previously generated content. *Data:* driving and street-scene video. *Finding:* explicit warping and 3D memory improve consistency. *Limitation:* flow and 3D structure are estimated, and the generator is heavy. *Relevance:* the closest analogue of EPAM's goal (consistent appearance on revisit); StructRender keys memory on engine identity instead of reconstructed geometry.

Richter et al., "Enhancing Photorealism Enhancement," IEEE TPAMI 2023 [14]. *Problem:* make rendered game frames look like real photographs. *Method:* an image-enhancement network conditioned on the game's intermediate G-buffers, trained adversarially against real-image datasets with a perceptual discriminator. *Data:* frames from GTA V paired with buffers, plus real datasets such as Cityscapes. *Finding:* G-buffer conditioning yields stable, geometry-consistent enhancement. *Limitation:* a photorealistic target domain backed by large real datasets; heavy network. *Relevance:* strongest evidence that engine buffers make neural restyling stable; StructRender targets artistic styles with no real-image domain, hence the teacher.

Ruder et al., "Artistic Style Transfer for Videos," GCPR 2016 [15] and **Lai et al., "Learning Blind Video Temporal Consistency," ECCV 2018** [16]. *Problem:* flicker when per-frame stylisation is applied to video. *Method:* a temporal loss on optical-flow-warped previous outputs with occlusion masks (Ruder); a post-processing network trained with short- and long-term temporal losses (Lai). *Data:* natural video (DAVIS and Videvo for Lai et al.). *Finding:* temporal losses with occlusion masking substantially reduce flicker. *Limitation:* depends on estimated flow; optimisation in Ruder et al. is slow. *Relevance:* StructRender's masked temporal loss follows the same principle, with the engine's analytic motion and an engine-derived disocclusion mask.

Kim et al., "GameGAN," CVPR 2020 [3]; **Valevski et al., "GameNGen," ICLR 2025** [4]; **Alonso et al., "DIAMOND," NeurIPS 2024** [5]; **Bruce et al., "Genie," 2024** [6]. *Problem:* learn a playable game simulator from observations. *Method:* a GAN with an explicit memory module and a separate rendering engine (GameGAN); diffusion models conditioned on past frames and actions (GameNGen, DIAMOND); a foundation world model with latent actions learned from unlabelled Internet gameplay video (Genie). *Data:* recordings of Pac-Man and VizDoom (GameGAN), agent play of DOOM (GameNGen), Atari 100k games (DIAMOND), and large Internet 2D-platformer video (Genie). *Finding:* generative models can produce interactive, playable footage. *Limitation:* game state is implicit in pixels or latents and can drift; long-horizon memory is limited. *Relevance:* the motivating contrast for StructRender, which keeps the engine authoritative.

Zhang et al., "ControlNet," ICCV 2023 [9]. *Problem:* add spatial control to a pretrained text-to-image diffusion model without damaging it. *Method:* a trainable encoder copy joined through zero-initialised convolutions. *Data:* paired condition/image sets for several conditions, including depth. *Finding:* robust conditioning on depth, edges and pose with small datasets. *Limitation:* tens of denoising steps per image; no temporal consistency. *Relevance:* the teacher is SD 1.5 with a depth ControlNet driven by the engine's own depth. The zero-initialisation idea is also reused in StructRender's temporal fusion and FiLM layers, so each new mechanism starts as an exact no-op.

Table 2.1: Comparison of related work

Reference	Year	Methodology	Dataset	Key findings	Limitations (for this project)
Nalbach et al. [35]	2017	CNN on G-buffers → screen-space shading	Rendered 3D scenes	Engine buffers suffice for learned shading	Single frame; photographic goal

Reference	Year	Methodology	Dataset	Key findings	Limitations (for this project)
Chaitanya et al. [32]	2017	Recurrent denoising autoencoder + aux. buffers	Path-traced sequences	Recurrence improves temporal stability	Reconstruction, not restyling
Xiao et al. [33]	2020	Motion-vector warping of past frames + CNN	Rendered game-like scenes	Real-time multi-frame reconstruction	Engine look only
Isola et al. [20]	2017	Conditional GAN, U-Net generator	Paired sets (e.g. Cityscapes, facades)	General structure-to-image translation	Needs paired data; no temporal model
Park et al. [36]	2019	Spatially-adaptive normalisation	COCO-Stuff, ADE20K, Cityscapes	Layout-faithful synthesis	Paired data; per-frame
Wang et al. [37]	2018	Sequential generator + flow warping	Street-scene video	Temporally coherent video synthesis	Estimated flow; heavy model
Mallya et al. [38]	2020	Guidance from 3D point cloud of past output	Street-scene video	Consistency on revisits	Needs reconstructed 3D; heavy
Richter et al. [14]	2023	G-buffer-conditioned enhancement, adversarial	GTA V + real datasets	Stable photorealism enhancement	Real-image domain; not artistic
Ruder et al. [15]	2016	Flow-warped temporal loss, occlusion masks	Natural video	Removes stylisation flicker	Optimisation-based; slow
Lai et al. [16]	2018	Post-hoc network with temporal losses	DAVIS, Videvo	Task-agnostic consistency	Estimated flow; extra pass
Kim et al. [3]	2020	GAN simulator with memory module	Pac-Man, VizDoom	Learned playable simulation	State implicit; drift
Valevski et al. [4]	2025	Diffusion next-frame model	Agent play of DOOM	Interactive diffusion game	No authoritative state
Alonso et al. [5]	2024	Diffusion world model for RL	Atari 100k	Visual detail aids agents	State implicit
Bruce et al. [6]	2024	Latent-action foundation world model	Internet 2D-platformer video	Playable worlds from video	No engine truth; drift
Zhang et al. [9]	2023	Zero-initialised control branch for diffusion	Paired condition/image sets	Depth/edge-controlled generation	Slow; no temporal consistency

2.2 Research Gaps

Table 2.2 links the gaps to the literature and to the way this project addresses them. Gaps G1–G4 follow directly from the reviewed work. Gaps G5–G7 are opportunities identified by the project team and are labelled as such.

Table 2.2: Research gaps and how StructRender addresses them

Gap	Evidence	Type	How this project addresses it
G1. Generative game models do not own game state, so world facts and appearance can drift	[3], [4], [5], [6]	Literature	Engine is the single source of truth; the network renders appearance only
G2. G-buffer-conditioned enhancement targets photorealism with real-image data	[14], [35]	Literature	Artistic styles supervised by an offline diffusion teacher
G3. Diffusion quality, but many network evaluations per image	[17], [8], [9]	Literature	Diffusion used offline only; a one-pass CNN runs live
G4. Per-frame stylisation flickers; consistency methods rely on estimated flow	[15], [16], [37]	Literature	Engine motion vectors and an engine-derived disocclusion mask
G5. Teacher targets may invent geometry; no standard acceptance test for aligned stylised targets	Project measurement (KB Decisions 32, 40)	Team-identified	Four pre-registered alignment gates; per-frame exclusion
G6. Persistent per-object appearance keyed by engine identity is unexplored	[38]	Team-identified	EPAM tested against a built-in no-memory control
G7. Live output rarely proven identical to the evaluated model	Project observation	Team-identified	Link-by-link equivalence chain from PyTorch to the game

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Architecture in a Nutshell

Figure 3.1 shows the complete system. It has an offline part (top two rows), which produces a validated dataset and a trained model, and a runtime part (bottom), which runs once per game frame inside Godot.

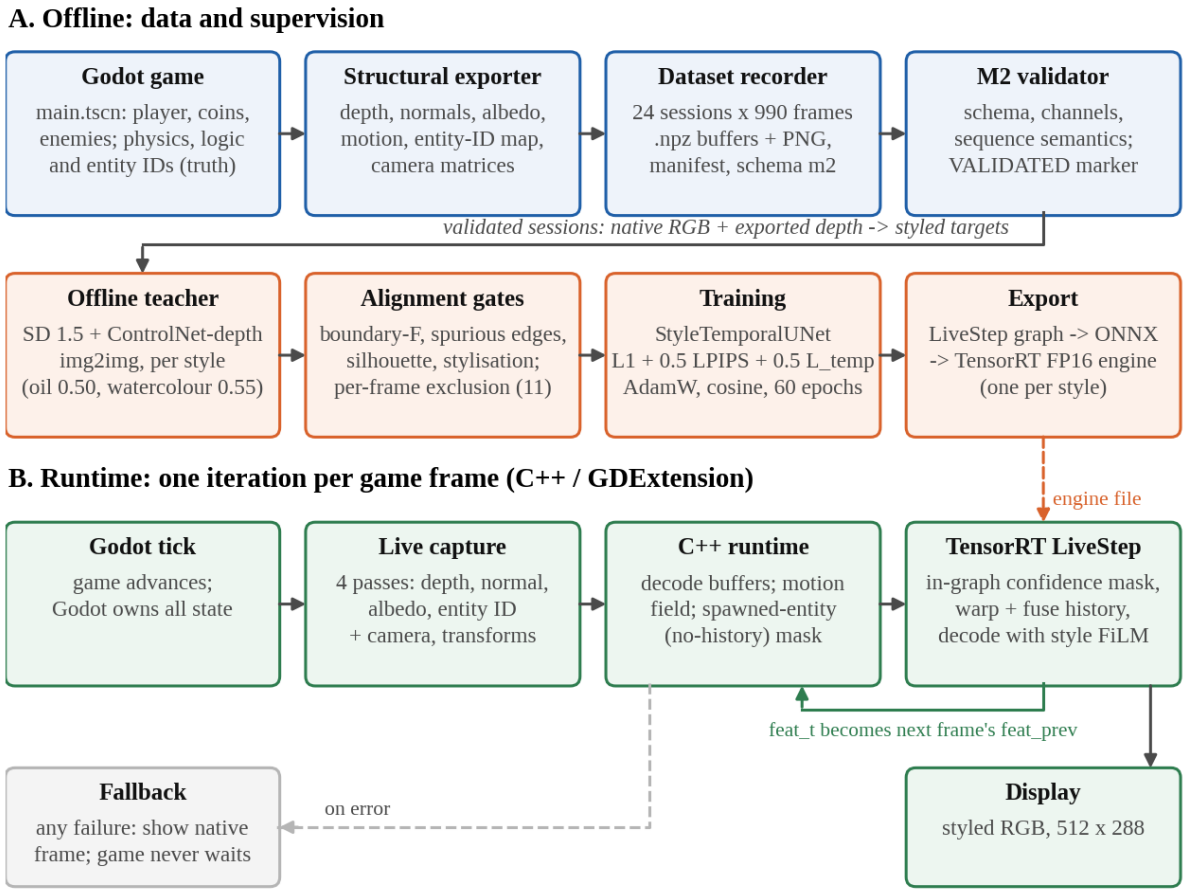


Figure 3.1: System architecture: offline data and supervision, and the runtime loop

The blocks are as follows.

- **Godot game (input source).** The playable arena collector (`main.tscn`): a player collects coins while enemies spawn and chase. Godot owns physics (60 Hz), logic and entity identity. The neural side never writes game state.
- **Structural exporter (data acquisition).** Override-shader passes re-render the current frame as depth, normals and albedo; an entity-ID pass labels each pixel with the visible entity. Motion vectors are computed analytically from camera and entity transforms

(backward, in normalised screen UV). Camera matrices and lifecycle events (spawn, visible, hidden, destroy) are also recorded.

- **Dataset recorder and validator.** Scripted play drives the game through its own input actions. Each recorded frame advances exactly one physics step, and the scene tree is paused while the passes render, so all buffers show the same instant. The validator checks schema, channel integrity and sequence semantics, and writes a VALIDATED marker only for clean sessions.
- **Offline teacher.** Stable Diffusion 1.5 with a depth ControlNet converts the native frame into a styled target, conditioned on the engine's exported depth (Section 3.1.2).
- **Alignment gates.** Four measurements reject targets that lose structure, invent edges, damage entity silhouettes, or fail to stylise. A frame that fails any gate in either style is excluded from training and testing.
- **Training.** The renderer (StyleTemporalUNet, Section 3.1.3) learns to map structure, with one frame of engine-warped history, to the teacher target for the selected style.
- **Export.** A single-step graph (*LiveStep*) containing the network and the confidence mask is exported to ONNX and compiled by TensorRT into one FP16 engine per style, on the target laptop.
- **Runtime loop.** Every frame, Godot captures four cheap passes. The C++ GDExtension decodes them, computes the motion field and the mask of newly spawned entities, and runs the TensorRT step with the previous frame's features. It then displays the result and keeps the new features for the next frame. Any failure shows the native frame instead, so the game never waits on the renderer.

3.1.1 Dataset Description

Two datasets were recorded with the same exporter, recorder and validator; both were generated by the project, so there is no external source, URL or DOI. The **game dataset** is the headline dataset: it is recorded from the actual playable game and is the one the deployed model was trained and tested on. The **seed-scene dataset** (ladder v3) was recorded earlier from an arena scene built in code; it was used for the ablation ladder (Section 4.3.2). Table 3.1 summarises both.

Table 3.1: Dataset summary

Property	Game dataset (deployed model)	Seed-scene dataset (ablation ladder)
Source	Scripted play of the playable game main.tscn through its own input actions	Arena scene built by the seed recorder
Sessions	24 recorded, 24 valid	30 recorded, 29 valid (1 rejected: non-finite motion)
Frames per session	990 (one physics step each, 16.5 s of game time)	990
Total frames	23,760	28,710 (valid sessions)
Resolution	512 × 288 (16:9)	512 × 288
Split (sessions)	18 train / 3 validation / 3 test	21 / 4 / 4
Split (frames)	17,820 / 2,970 / 2,970	20,790 / 3,960 / 3,960
Targets (styles)	oil_impasto (strength 0.50) and watercolor_bold_s055 (0.55)	oil_impasto (0.50); watercolour added for EPAM runs
Excluded by alignment gates	11 frames (0.05 %)	Whole-session rule
Storage	Per frame: .npz buffers + 8-bit PNG + camera JSON; 2.9 GB structural data	Same format
Licence	Project-generated data; teacher weights under their own licences [18], [19]	As for game dataset

Inputs and targets. Table 3.2 lists what each frame record contains and how it is used. The task is dense regression from structure to a styled RGB image, so there are no class labels; the *label* of a frame is the teacher's styled image for the chosen style. Targets are stored in a sidecar dataset per style, referencing the immutable structural session by session and frame identifier.

Table 3.2: Contents of one recorded frame and their use

Field	Type and shape	Origin	Use
Native RGB	uint8, 288 × 512 × 3 (PNG)	Godot viewport, no anti-aliasing	Teacher input; "do-nothing" anchor in evaluation
Depth	float16, metres; background = +∞	Override-shader pass	Network input (inverse depth); teacher conditioning; confidence mask
Normals	float16, 3 channels	Override-shader pass	Network input; confidence mask
Albedo	float16, 3 channels	Override-shader pass	Network input
Motion	float16, 2 channels, backward UV	Analytic, from transforms	Warping history and the mask; never an input channel
Entity ID	int32 per pixel	Override-shader pass	Addressing only (spawned-entity mask, alignment gates); never an input channel
Camera	view and projection matrices	Engine	Motion computation; live capture
Lifecycle	events with EntityKey (session, instance, generation)	Entity adapter	Sequence validation; memory keys for EPAM

Annotation methodology. No manual annotation is involved. Structural channels are exact engine values; targets are generated by the offline teacher and then accepted or rejected by automatic gates.

Limitations and biases. The game uses flat, untextured grey geometry with plain lighting, so the targets are mild stylisations. Only a few entity types occur (player, coins, enemies), so methods that should matter with many entities and recycled IDs are not stressed. The 11 excluded frames are the views on which the teacher was hardest to keep aligned (busy edges, thin silhouettes); removing them slightly under-represents such views in both training and test [13]. Sessions were recorded with a seeded wandering route, so play style is not that of a human player.

3.1.2 Preprocessing

Only steps that are actually implemented are described here, in pipeline order.

(a) Validation and cleaning. The validator rejects any session with schema errors, missing or partially dropped channels, non-finite values, or inconsistent lifecycle sequences. One seed-scene session was rejected for non-finite motion at frame 616 before it reached training. A frame-diversity check also warns when a session is dominated by repeated frames.

(b) Teacher target generation. For each validated frame and style, SD 1.5 img2img with a depth ControlNet generates the target from the native frame, conditioned on the engine depth (Table 3.3). Generation is seed-pinned and cached by content hash, so identical inputs are never regenerated differently.

Table 3.3: Offline teacher configuration

Parameter	oil_impasto	watercolor_bold_s055
Base model	SD 1.5 [18], rev. 451f4fe1	same
Control model	ControlNet depth [19], rev. 35e42a3e	same
Conditioning	engine-exported depth buffer	same
Inference steps / guidance	24 / 7.5	24 / 7.5
ControlNet conditioning scale	1.6	1.6
img2img strength	0.50	0.55
Seed	20260906	20260930
Prompt (abridged)	thick oil painting, heavy impasto brush strokes, warm ochre and deep blue palette	bold watercolour on cream paper, saturated ultramarine and viridian washes, colour blooms

(c) Alignment gates. Each target is scored against the engine's own structure with four gates whose thresholds were fixed before the game data existed (Table 3.4). The structural edge map is built from depth discontinuities and entity-ID boundaries. The boundary F-measure is the harmonic mean of edge precision and recall within a 2-pixel tolerance:

$$F = 2 \cdot P \cdot R / (P + R) \tag{3.1}$$

where P is the fraction of target edges near a structural edge and R the fraction of structural edges reproduced. Gate 4 exists because gates 1–3 are trivially passed by a teacher that changes nothing; it measures the L1 distance between 32-bin per-channel colour histograms of target and native frame (range 0–2).

Table 3.4: Alignment gates (thresholds fixed before the data)

Gate	Measures	Threshold
1. Boundary F-measure	structural edges preserved (2 px tolerance)	≥ 0.60
2. Spurious-edge ratio	strong target edges with no structural support (invented geometry)	≤ 0.35
3. Silhouette support	each entity of ≥ 200 visible px keeps its outline	≥ 0.60
4. Stylisation (histogram L1)	target actually differs from the native frame	≥ 0.15

Style strength was selected by a pre-registered rule: take the strongest setting with zero failures on a 45-frame probe, then require it to pass a full 990-frame session. Oil passed at 0.50. The original watercolour failed on the game's flat grey surfaces (mostly "not stylised"), and a bolder watercolour prompt passed at 0.55 [13].

(d) Per-frame exclusion and leakage prevention. A frame that fails any gate in either style is excluded from both styles, together with every temporal triple or history pair that touches it; a frame whose predecessor is excluded is scored with no history. This removed 11 of 23,760 frames (0.05 %). The split is by **whole session** and was fixed by position before any target, model or metric existed. Validation and test sessions never contribute training frames, so no frame of a test session can leak into training. The test split was scored once, at the end.

(e) Input encoding. The network input is 7 channels: inverse depth (1), normals (3) and albedo (3). Depth in metres (d) is mapped to a bounded range, with the background ($d = +\infty$) mapping to 0:

$$x_{depth} = 1 / (1 + d) \tag{3.2}$$

Motion vectors and entity IDs are deliberately **not** input channels. Motion has an explicit role (warping), and entity IDs are addresses, not quantities: a numeric ID channel would make a

recycled ID look like the entity it replaced [13]. Targets are 8-bit sRGB scaled to [0, 1]. No data augmentation or class balancing is used: augmentation would break the exact correspondence between structure, motion and target that the temporal model relies on, and the task has no classes.

(f) Temporal samples and the confidence mask. A training sample is a run of three consecutive frames (t−2, t−1, t) from one session. Rendering frame t−1 *with its own history* needs t−2, so the temporal loss compares the model's history-informed outputs. For each step a per-pixel confidence mask M_t decides which pixels may trust history. With $W(\cdot, V_t)$ the backward warp by the engine motion field:

$$e_D = | D_t - W(D_{t-1}, V_t) | , \quad e_N = 1 - N_t \cdot W(N_{t-1}, V_t) \tag{3.3}$$

$$M_t = \exp(-\alpha\, e_D - \beta\, e_N) \cdot H_t , \quad \alpha = \ln 2 / 0.175 \text{ m} , \quad \beta = \ln 2 / (1 - \cos 30°) \tag{3.4}$$

where D is depth, N the unit normal, and $H_t \in \{0, 1\}$ a set of hard zeros: no previous frame, motion leaving the screen, background at time t or t−1, and pixels of entities that did not exist in the previous frame (newly spawned). The 0.175 m depth scale was calibrated as the optimal separation (Youden index) between depth steps within surfaces and across object silhouettes, measured on the data. Figure 3.2 shows the structural inputs, motion and confidence mask for one sealed test frame.

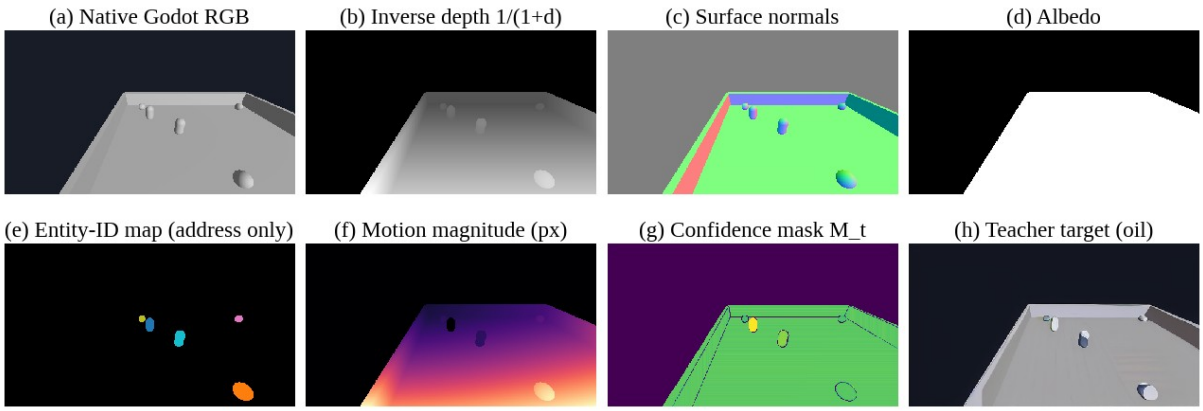


Figure 3.2: Structural inputs, motion and confidence mask for one test frame

3.1.3 Encoder Layer (Feature-Extraction and Rendering Network)

The heading has been adapted to the project's architecture: the network is a convolutional encoder–decoder, and its encoder is the feature-extraction component. The deployed model is **StyleTemporalUNet** (Figure 3.3; Table 3.5). It has 8,394,915 parameters.

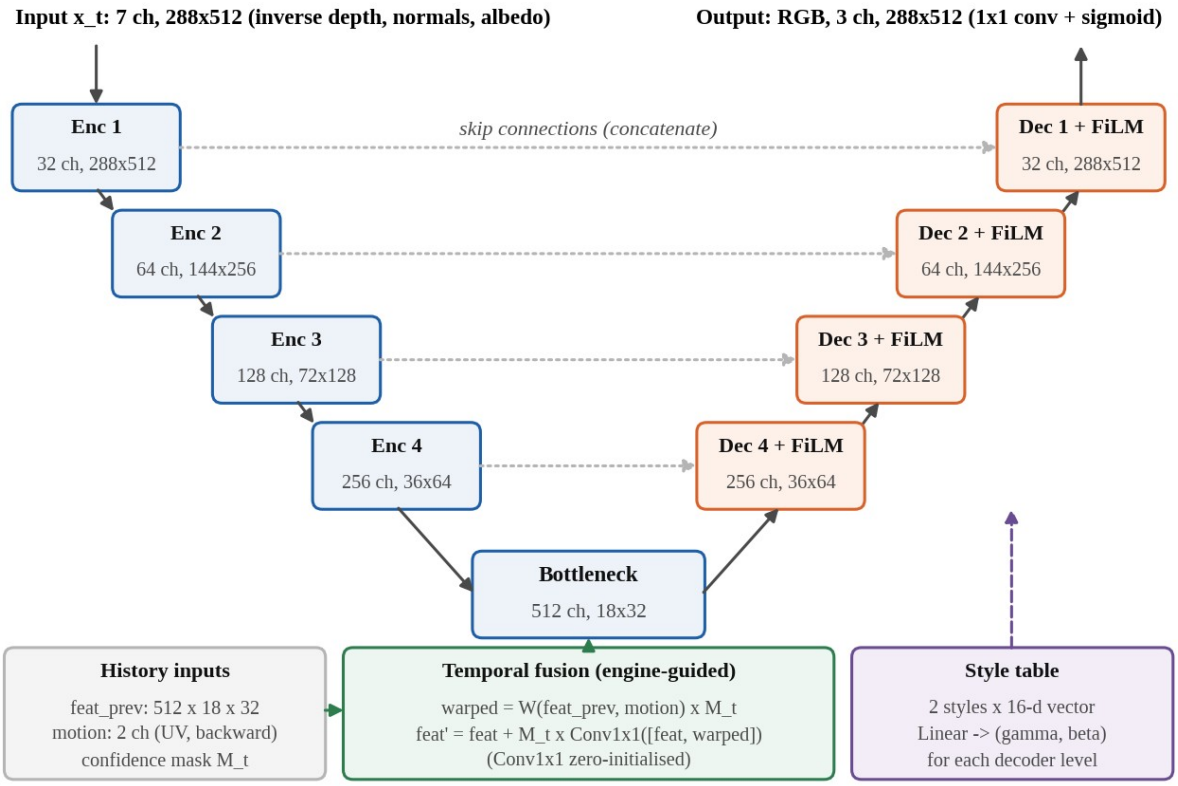


Figure 3.3: StyleTemporalUNet: style-free encoder, temporal fusion, FiLM decoder

Purpose and input. The encoder turns the 7-channel structural frame x_t (288 × 512) into a hierarchy of features. It reads only engine structure, never the style. Each frame's bottleneck features are therefore identical whichever style is displayed, so the same features can serve as history across a live style switch.

Internal structure. Four encoder levels each apply two 3 × 3 convolutions, each followed by group normalisation (8 groups) [21] and ReLU, then 2 × 2 max pooling. The channel widths are 32, 64, 128 and 256 at 288 × 512, 144 × 256, 72 × 128 and 36 × 64. A bottleneck of two further convolution blocks produces 512 channels at 18 × 32. Group normalisation was chosen over batch normalisation because it is independent of batch size and exports to ONNX without running statistics.

Temporal fusion. At the bottleneck, the previous frame's features F_{t-1} are warped to the current frame with the engine motion field, masked by the area-pooled confidence mask, and fused through a 1 × 1 convolution whose weights start at zero:

$$F'_t = F_t + M_t \odot \mathrm{Conv}_{1\times1}([\,F_t\,,\, M_t \odot W(F_{t-1}, V_t)\,]) \tag{3.5}$$

Zero initialisation makes an untrained temporal model exactly equal to the single-frame U-Net, so the rung adds a mechanism without changing the starting point. Where $M_t = 0$ (disoccluded or new pixels) the history term vanishes and the frame is rendered from structure alone.

Decoder with style modulation. Each decoder level upsamples by 2 (nearest neighbour), concatenates the matching encoder skip connection, and applies two convolution blocks. Then FiLM [26] modulates every channel with a scale and shift computed from the selected style's learned 16-dimensional vector s:

$$h' = h \odot (1 + \gamma_s) + \beta_s , \quad [\ \gamma_s\ ;\ \beta_s\ ] = A_l\, s + b_l \tag{3.6}$$

The projections A_l, b_l are zero-initialised, so an untrained two-style model is exactly the single-style Model B, and the difference between styles is learned. The style table itself is initialised from a truncated normal distribution so that gradients reach the FiLM weights from the first step. A final 1 × 1 convolution and a sigmoid produce RGB in [0, 1].

Table 3.5: Layer summary of StyleTemporalUNet (parameters counted from the model)

Block	Output (channels × H × W)	Parameters
Input: inverse depth, normals, albedo	7 × 288 × 512	–
Encoder level 1	32 × 288 × 512	11,424
Encoder level 2	64 × 144 × 256	55,680
Encoder level 3	128 × 72 × 128	221,952
Encoder level 4	256 × 36 × 64	886,272
Bottleneck	512 × 18 × 32	3,542,016
Temporal fusion (1 × 1 conv, zero-init)	512 × 18 × 32	524,800
Decoder level 4 (+ FiLM)	256 × 36 × 64	2,360,832 + 8,704
Decoder level 3 (+ FiLM)	128 × 72 × 128	590,592 + 4,352
Decoder level 2 (+ FiLM)	64 × 144 × 256	147,840 + 2,176
Decoder level 1 (+ FiLM)	32 × 288 × 512	37,056 + 1,088
Head (1 × 1 conv + sigmoid)	3 × 288 × 512	99
Style table (2 styles × 16)	–	32
Total		**8,394,915**

Contribution to the output. The encoder carries geometry and layout; the skip connections keep edges sharp; the temporal fusion stabilises the output against the teacher's frame-to-frame variation; and the FiLM layers carry the palette and texture of each style.

Training objective. The objective combines reconstruction, perceptual and temporal terms with weights fixed for every model in the project:

$$L = \| \hat{I}_t - I_t \|_1 + 0.5 \cdot \mathrm{LPIPS}(\hat{I}_t, I_t) + 0.5 \cdot L_{temp} \tag{3.7}$$

$$L_{temp} = \Sigma_p\, M_t(p) \cdot | \hat{I}_t(p) - W(\hat{I}_{t-1}, V_t)(p) | \ /\ \Sigma_p\, M_t(p) \tag{3.8}$$

where I_t is the teacher target, $\hat{I}_t$ the prediction and p a pixel. The temporal term uses the prediction on both sides (a term built from two teacher images would have no gradient) and the

same mask as the evaluation metric. A frozen or blurred output would score zero on this term, which is why the reconstruction and perceptual terms, and the sharpness gate in evaluation, are always present.

Comparison models. For the ablation (Section 4.3.2), Model A is the same U-Net without history, and StructFormer (Models C–F) replaces the encoder–decoder with a convolutional stem, window-attention encoder [34], window-attention decoder and convolutional RGB head. Model E adds entity-masked cross-attention over entity tokens, and Model F adds the EPAM memory. Every rung was trained at parameter parity (0.9–1.1×) with the same data, seed, objective, early stopping and selection rule [13].

3.2 Hardware Specifications

All development, dataset recording, teacher generation, training, evaluation and live deployment reported here ran on a single laptop (Table 3.6). The project plan allowed rented cloud GPUs for final runs, but the project's logs record no cloud run behind any reported result, so development, training and deployment hardware are the same machine. The TensorRT engines were built on this GPU, as TensorRT engines are specific to the GPU and software version they are built with [12].

Table 3.6: Hardware used for all experiments

Component	Specification
Machine	Laptop — make and model [DETAILS REQUIRED]
CPU	AMD Ryzen 9 8940HX, 16 cores / 32 threads
System memory	30 GiB visible to the operating system
GPU	NVIDIA GeForce RTX 5060 Laptop GPU (Blackwell, compute capability 12.0)
GPU memory	8 GB (8,151 MiB reported), shared with the desktop display
GPU driver	610.57.04
Storage	2 × 1 TB NVMe SSD (Micron MTFDKBA1T0QGN; WD PC SN560)
Operating system	Omarchy (Arch Linux based), Linux kernel 7.2.5

Role of the hardware. The CPU runs Godot, dataset decoding (8 data-loader workers for temporal training) and the C++ motion computation. The GPU runs teacher generation, training under bfloat16 mixed precision, evaluation and TensorRT inference. The 8 GB of video memory was a binding constraint: training shared the GPU with the desktop, so the batch was reduced to 4 with gradient accumulation of 2 (effective batch 8). `expandable_segments` memory allocation was enabled to avoid starving the display.

3.3 Software Specifications

Table 3.7: Software environment (versions read from the environment)

Category	Software and version	Purpose
Game engine	Godot 4.7.2 (Forward+ renderer) [7]	Game, physics, structural export, live loop
Engine extension	C++17, godot-cpp [39], GCC 16.2.1, CMake 4.4.4	StructRenderer GDExtension and C++ runtime
Language (ML)	Python 3.10.21 (conda environment)	Data pipeline, training, evaluation, export
Deep learning	PyTorch 2.13.0 (CUDA 13.2 build) [40], torchvision 0.28.0	Models, training, evaluation
Inference	TensorRT 11.2.1.2 [12]; ONNX 1.22.0 [11]; ONNX Runtime 1.23.2	Export, engine building, runtime
GPU stack	NVIDIA driver 610.57.04; CUDA toolkit 13.4 (system)	GPU compute
Teacher	Diffusers 0.40.0 [41], Transformers 5.16.1, Accelerate 1.14.0	SD 1.5 + ControlNet target generation
Metrics	lpips 0.1.4 (AlexNet trunk) [30]; project SSIM/PSNR code	Perceptual and pixel metrics
Scientific stack	NumPy 2.2.6, SciPy 1.15.3, Pillow 12.3.0	Arrays, image I/O
Visualisation	matplotlib 3.10.9; ffmpeg (demo video)	Figures, demo
Testing	pytest 9.1.1	Automated test suite
Version control	Git, GitHub (releases for weights)	Reproducibility

Integration. Python code records nothing itself. Godot scripts produce the dataset, and Python reads it for teacher generation, training and evaluation. A trained checkpoint is exported once to ONNX, and TensorRT compiles it on the target machine. The C++ GDExtension loads the compiled engines and is called from GDScript every frame. Python and C++ use the same TensorRT build, so engines and numerical checks are shared between them.

CHAPTER 4: PERFORMANCE EVALUATION

4.1 Experimental Setup

The environment is described in Sections 3.2–3.3, and the data, splits and preprocessing in Section 3.1. Table 4.1 gives the configuration of the deployed two-style model (configuration file `configs/game/model_b_two_style.json`). The ablation rungs used the same optimiser, schedule, objective, early-stopping rule and selection rule on the seed-scene dataset.

Table 4.1: Training and evaluation configuration of the deployed model

Item	Setting
Model	StyleTemporalUNet, base width 32, 8,394,915 parameters
Data	Game split v1: 18 train sessions, both styles; 35,526 training triples; 5,936 validation frames
Objective	1 · L1 + 0.5 · LPIPS(AlexNet) + 0.5 · L_temp (mask-weighted) — Eq. (3.7)
Optimiser	AdamW [42], learning rate 2×10^{-4}, weight decay 1×10^{-4}
Schedule	Cosine annealing [43] over optimiser steps, no warm-up
Batch	4 × gradient accumulation 2 = effective 8
Precision	bfloat16 automatic mixed precision
Epochs	60 maximum; validation every 2 epochs
Early stopping	patience 6 evaluations, minimum relative improvement 0.001 (not triggered)
Checkpoint selection	lowest validation L1, averaged over both styles
Random seed	20260905 (same seed for every rung)
Wall time	29.2 h (105,249 s); best epoch 49, validation L1 0.007435
Test protocol	3 sealed sessions, 2,968 frames per style, each rendered with its real previous frame; flicker on 2,451 moving pairs
Inference	TensorRT FP16 engine per style; FP32 confidence mask; 512 × 288

Baselines and scenarios. (i) The ablation ladder on the seed scene (Models A–F and control N; Table 4.2). (ii) The deployed model compared with an oil-only Model B trained on the same game data, and with the unstyled native frame as a do-nothing anchor. (iii) The same model with and without history. (iv) The live game: speed, a 3,000-frame stability run, failure injection, and a manual play session.

Reproducibility measures. Thresholds were written into evaluator code or the Decision Log before results existed. Fixed seeds, configuration files and generated JSON reports are under version control. The test split was used once. Weights are published as GitHub releases. Data is not augmented and is loaded in a deterministic order. Seed variance was measured on the seed scene (three seeds for Models C and E) and used to set noise thresholds. The project's

automated test suite (548 tests, covering data validation, the alignment gates, warping and masking, model equivalence, export and the runtime contract) passed in full when re-run on 23 October 2026 (548 passed, 10 min 46 s). On the game, each model was trained once, so small differences between game models are not claimed as significant.

Table 4.2: Ablation ladder: each rung adds one idea

ID	Model	Backbone	Temporal	Entity use	EPAM	Parameters
A	CNN baseline	U-Net	No	No	No	7,853,763
B	CNN temporal	U-Net	Yes	No	No	8,378,563
C	StructFormer	StructFormer	No	No	No	7,775,467
D	StructFormer temporal	StructFormer	Yes	No	No	8,236,747
E	StructFormer entity-aware	StructFormer	No	Masked attention	No	7,053,547
N	Naive control	StructFormer	No	Raw ID channel	No	7,777,627
F / F0	EPAM / no-memory twin	StructFormer	No	Masked attention	Yes / No	E + 16

4.2 Evaluation Metrics

The task is dense image regression with a temporal and a real-time requirement, so classification metrics (accuracy, precision, recall, F1, confusion matrix, ROC-AUC, mAP, IoU) do not apply. The metrics used are defined below. In all of them $\hat{I}$ is the prediction, I the teacher target, both in [0, 1], N is the number of pixel values, and averages are over test frames.

Mean absolute error (L1). The average per-pixel colour error. It is the main training term and the checkpoint selection criterion. Lower is better. Like any pixel error, it rewards blur under uncertain targets [31].

$$L1 = (1/N)\ \Sigma_i\ |\ \hat{I}_i - I_i\ | \tag{4.1}$$

Peak signal-to-noise ratio (PSNR). The logarithmic inverse of the mean squared error, in decibels; higher is better. A 1 dB gain is a 21 % reduction in MSE. It shares L1's preference for blur.

$$PSNR = 10 \cdot \log_{10}(\ 1 / MSE\ )\ ,\quad MSE = (1/N)\ \Sigma_i\ (\ \hat{I}_i - I_i\ )^2 \tag{4.2}$$

Structural similarity (SSIM) [28]. It compares local means μ, variances σ^2 and covariance σ_{xy} in 11 × 11 Gaussian windows (σ = 1.5); 1 means identical. C_1, C_2 are small stabilising constants. On this dark, low-contrast scene it compresses differences into the third decimal place.

$$SSIM(x, y) = (2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)\ /\ ((\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)) \tag{4.3}$$

LPIPS [30]. A weighted distance between normalised AlexNet feature maps; lower is more perceptually similar. Because LPIPS is also part of the training objective, it is never quoted alone as evidence.

Sharpness ratio. The high-pass energy of the prediction relative to the target, using squared Sobel gradients of luma *Y*, summed over all test frames. It is a blur guard. A ratio of 0.90 corresponds to roughly a 1-pixel Gaussian blur of the target (blurring the targets gives 0.977, 0.897 and 0.761 at $\sigma = 0.5$, 1.0 and 1.5). It does not prove quality on its own, since the unstyled native frame also passes it.

$$S = \Sigma_{\text{frames}} \Sigma_p \| \nabla_{\text{Sobel}} Y(\hat{I})(p) \|^2 \;/\; \Sigma_{\text{frames}} \Sigma_p \| \nabla_{\text{Sobel}} Y(I)(p) \|^2 \tag{4.4}$$

Colour-histogram travel (style acquisition). $h(\cdot)$ is the concatenated 32-bin histogram of the three channels, normalised to sum to 1 per image, and I_{nat} the native frame. The ratio measures how far the model moves the palette away from the plain game, relative to how far the teacher moves it. The gate is ≥ 0.80. It is one-sided: blur keeps a shifted palette, which is why it is paired with sharpness.

$$T(x) = \| h(x) - h(I_{\text{nat}}) \|_1 , \quad \text{travel ratio} = T(\hat{I}) / T(I) \tag{4.5}$$

Temporal error (flicker). The mean difference between a frame and the previous frame warped by the engine's motion, over the pixels the engine-derived mask trusts. It is evaluated only on pairs whose median motion is at least 1 pixel. Lower is steadier. A frozen or blurred output has low temporal error, so it is always reported with sharpness.

$$TE = \Sigma_p M_t(p) \, | I_t(p) - W(I_{t-1}, V_t)(p) | \;/\; \Sigma_p M_t(p) \tag{4.6}$$

Noise threshold for one run against one run. With sd_{rel} the relative standard deviation measured across seeds, a difference between two single runs is claimed only if it exceeds two standard deviations of a difference:

$$\text{threshold} = 2 \cdot \sqrt{2} \cdot \text{sd}_{\text{rel}} \tag{4.7}$$

Runtime metrics. Frame time is the wall-clock time of one complete live-loop iteration (capture, motion, inference, upload and display), and frames per second is 1000 / median frame time (ms). Engine latency is the median of 500 standalone TensorRT executions. The fallback rate is the fraction of styled frames for which the native frame had to be shown. Medians are used because single frames can stall on the operating system or desktop compositor.

Equivalence. The maximum absolute difference between outputs of two implementations given identical inputs (for example PyTorch against TensorRT, or GDScript motion against C++ motion).

4.3 Results and Analysis

4.3.1 Training Behaviour

Figure 4.1 shows the training run of the deployed model. The objective falls steadily over 60 epochs. Validation L1 decreases to its minimum of 0.007435 at epoch 49 and then flattens, so early stopping was not triggered, and the selected checkpoint is the epoch-49 model. The temporal term *rises* during training. This is expected: early in training the output is smooth and therefore trivially stable, and as the model learns the teacher's texture it acquires some of the teacher's frame-to-frame variation. That is why temporal error is reported with sharpness.

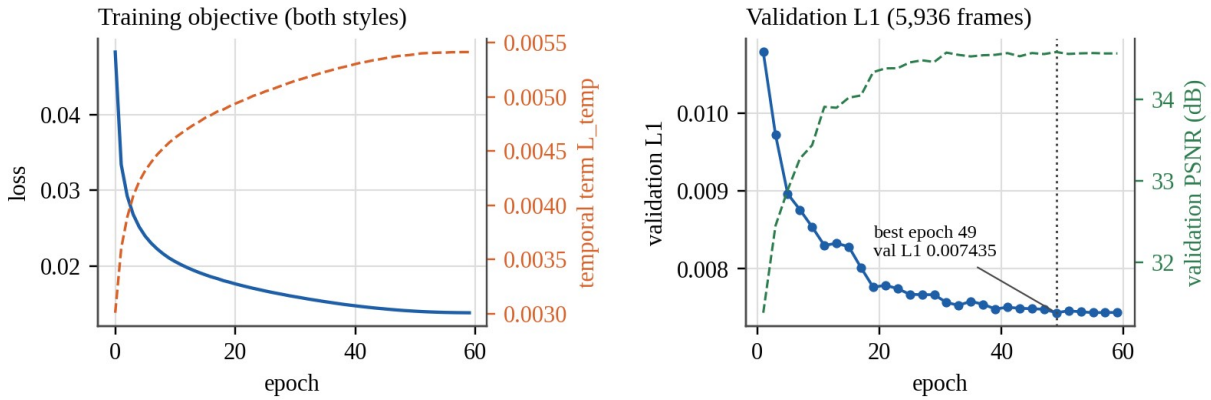


Figure 4.1: Training objective and validation L1/PSNR of the deployed model

4.3.2 Ablation Ladder on the Seed Scene

Table 4.3 and Figure 4.2 report the ladder on the same 3,960 sealed test frames (4 held-out sessions, oil style), scored by one evaluation script [13].

Table 4.3: Ablation ladder, sealed test split (3,960 frames)

Rung	L1	PSNR (dB)	SSIM	LPIPS	Sharpness	Parameters
A U-Net	**0.008947**	**34.041**	**0.9747**	**0.0459**	**0.912**	**7,853,763**
B U-Net + temporal	0.009296	33.643	0.9734	0.0489	0.897	8,378,563
C StructFormer	0.009174	33.858	0.9728	0.0466	0.853	7,775,467
D StructFormer + temporal	0.009336	33.555	0.9727	0.0485	0.858	8,236,747
E StructFormer + entity	0.009219	33.789	0.9726	0.0463	0.884	7,053,547
N raw-ID control	0.009054	33.865	0.9732	0.0459	0.880	7,777,627

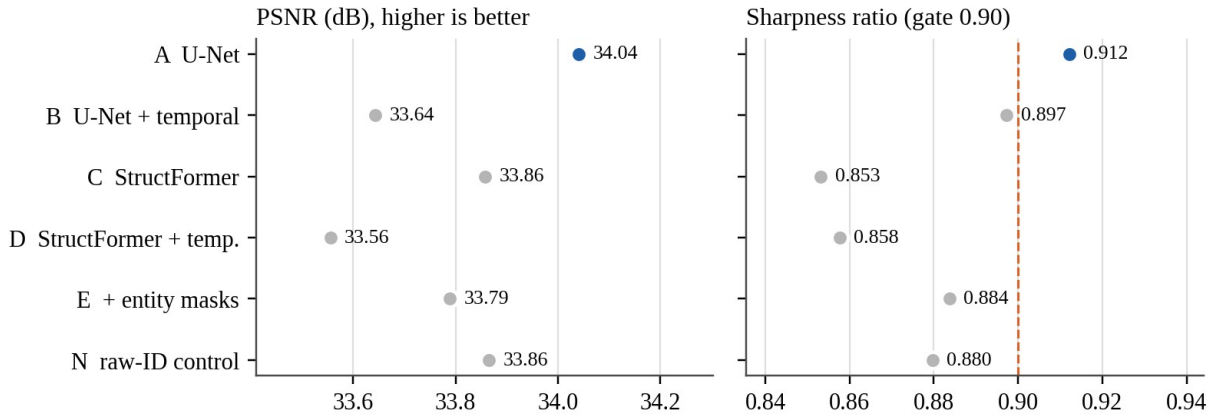


Figure 4.2: Ablation ladder: PSNR and sharpness on the sealed test split

The plain U-Net (A) is best on every image-quality metric, and it is the only rung that clears the 0.90 sharpness gate. StructFormer was expected to catch up with more data. Its sharpness was 0.833 at 644 training frames, 0.886 at 10,890 and 0.851 at 20,790. A projection of 0.898 at 20,790 frames had been recorded before the run, so the scaling hypothesis is refuted; the model is 0.990× the U-Net's parameter count, so capacity is not the explanation. At this scale and parameter count, the windowed transformer's inductive bias is worse suited than the CNN's.

Temporal conditioning reduces flicker on both backbones (Table 4.4, Figure 4.3). The verdict rule was fixed before measurement: improvement must exceed $2\sqrt{2} \cdot \mathrm{sd}_{\mathrm{rel}} = 3.85$ %, with $\mathrm{sd}_{\mathrm{rel}} = 1.36$ % from Model E's three seeds. B reduces temporal error by 15.2 % relative to A, and D by 10.7 % relative to C; both clear the threshold well. The cost is 1.8–3.9 % in per-frame L1. For a live renderer, flicker is the more visible defect, so Model B was chosen for the game. One caveat applies: B and D also train with the temporal loss, so the gain belongs to *history plus temporal loss*, not to the fusion architecture alone.

Table 4.4: Temporal error on 501 moving test pairs (lower is steadier)

Source	Temporal error	Change vs partner	Verdict (threshold 3.85 %)
Native engine	0.000241	–	–
Teacher targets	0.006265	–	–
A U-Net	0.003175	–	–
B U-Net + temporal	0.002692	−15.21 % vs A	Improves
C StructFormer	0.004191	–	–
D StructFormer + temporal	0.003741	−10.73 % vs C	Improves
E StructFormer + entity (3 seeds)	0.004172 / 0.004159 / 0.004264	–	–

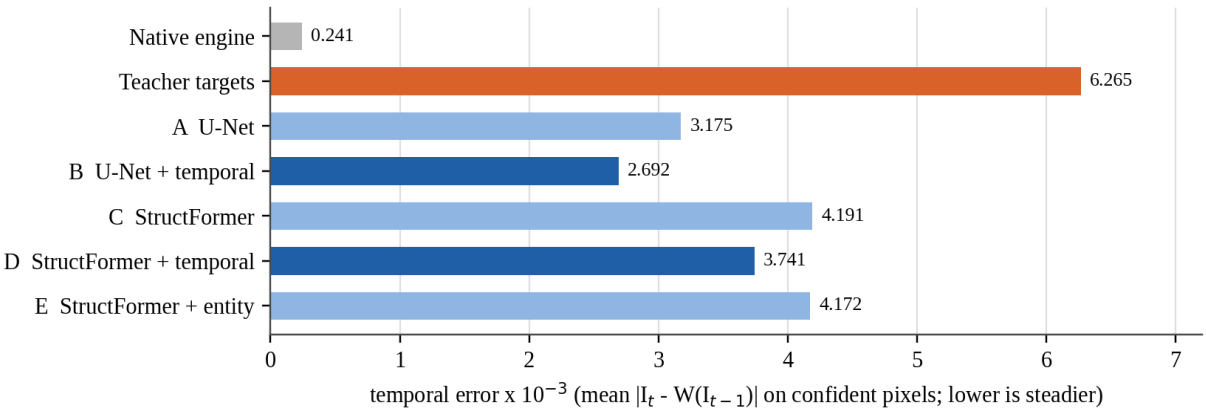


Figure 4.3: Temporal error of every rung against the engine and the teacher

Entity awareness and the naive control. Over three seeds, E is sharper than C by +0.015 at 2.5σ, in the direction predicted before training. However, the naive control N, which simply feeds the entity ID as a number, recovers about 87 % of that gain, and N and E are indistinguishable (sharpness difference −0.004 against a seed spread of 0.007). With only two or three stable entity IDs in this scene, exposing the IDs is enough. The structured design is not shown to be necessary here; showing it would need a scene with many entities and recycled IDs. The production rule (IDs are addresses, never input values) rests on lifecycle safety, not on this metric.

EPAM (objective O8). Model F (memory on, update rate γ = 0.9) and F0 (γ = 0, the built-in no-memory twin) share code, clips, styles, objective, schedule and selection rule. The pass threshold, $2\sqrt{2} \times 0.0073 = 0.0206$ in sharpness, was written into the evaluator before the run. The result (Table 4.5) is **negative**: F − F0 = −0.0127, inside noise and in the wrong direction. Three independent measurements agree that this data gives memory nothing to recover. Appearance residuals are uncorrelated across occlusion gaps (correlation ≤ 0.037 beyond 30 ticks). The reappearance metric does not degrade with time unseen. And anchoring the teacher's noise per entity does not create persistent appearance. The likeliest cause is the supervision: the diffusion teacher re-invents each object's look every frame, so there is no stable per-entity appearance in the targets for a memory to hold. The memory lifecycle requirements (retrieval after leaving view, no leakage after destruction, clearing on style change) are met and asserted by 17 tests.

Table 4.5: EPAM against its no-memory twin (sealed test, 3,912 clips, two styles)

Subset	Clips	F sharpness	F0 sharpness	F − F0	F L1	F0 L1
All	3,912	0.8829	0.8956	−0.0127	0.009064	0.009158
oil_impasto	1,956	0.8775	0.8809	−0.0034	0.009262	0.009291
watercolor	1,956	0.8882	0.9101	−0.0219	0.008866	0.009025
Memory can act	2,712	0.8634	0.8749	−0.0114	0.010280	0.010360

4.3.3 The Deployed Two-Style Model on the Game

Table 4.6 reports the held-out test results of the deployed model, alongside an oil-only Model B re-scored on the identical frames. Both styles clear the 0.90 sharpness gate. Sharing one model between two styles costs oil 0.10 dB PSNR and 0.0009 LPIPS, with identical sharpness. Because each game model was trained once and game-level seed variance was not measured, this difference is reported as a measurement, not as a significant effect.

Table 4.6: Game test sessions, 2,968 frames per style

Model, style	L1	PSNR (dB)	SSIM	LPIPS	Sharpness	Flicker: model / teacher
Two-style B, oil	0.00610	35.33	0.9807	0.0225	0.946	0.00566 / 0.01041
Two-style B, watercolour	0.00805	34.79	0.9786	0.0243	0.937	0.00614 / 0.01270
Oil-only B, oil (reference)	0.00606	35.43	0.9814	0.0216	0.946	0.00558 / 0.01041

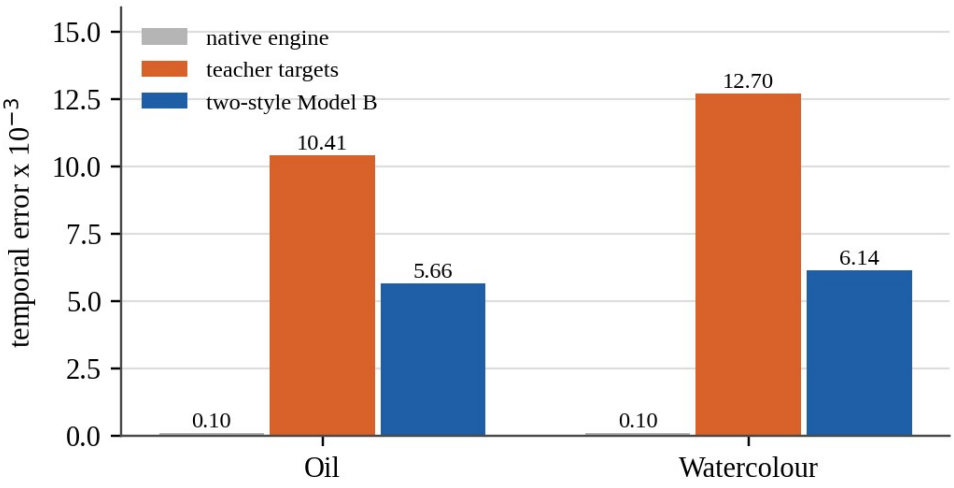


Figure 4.4: Flicker on the game's test sessions: engine, teacher and model

Steadier than its teacher. Figure 4.4 shows that the renderer flickers about half as much as the diffusion targets it learned from (0.54× for oil, 0.48× for watercolour), on 2,451 moving pairs. It is still about 57× the native engine's temporal error (0.000100). Temporal conditioning narrows this gap; it cannot close it while the targets themselves flicker.

The style switch is real, not a blend. On test session f1f9c484 (every 10th frame, 99 frames), each style's output is about 4× closer (in L1) to its own teacher than to the other (Table 4.7). The two outputs differ from each other by 0.0242, against 0.0260 between the two teachers, so the 16-dimensional style vector reproduces about 93 % of the inter-style difference. This is a sampled check, not a full-set metric.

Table 4.7: Mean L1 between each style's output and each style's targets (sampled)

Output	vs oil targets	vs watercolour targets
Oil style	0.0058	0.0257
Watercolour style	0.0247	0.0079

Style acquisition. Table 4.8 shows that both styles move the palette 98.7–99.5 % as far from the plain game as the teacher does (gate ≥ 80 %), and beat the do-nothing anchor (the native frame scored against the target) by 7.3–7.8 dB. Together with the sharpness results, this excludes both failure modes: an output that is sharp but unstyled, and one that is styled but blurred.

Table 4.8: Style acquisition on the game test frames

Model, style	Histogram travel ratio (≥ 0.80)	PSNR vs target	Do-nothing PSNR	Gain
Two-style B, oil	0.995 PASS	35.33	28.06	+7.27 dB
Two-style B, watercolour	0.987 PASS	34.79	26.96	+7.83 dB
Oil-only B, oil	0.980 PASS	35.43	28.06	+7.37 dB

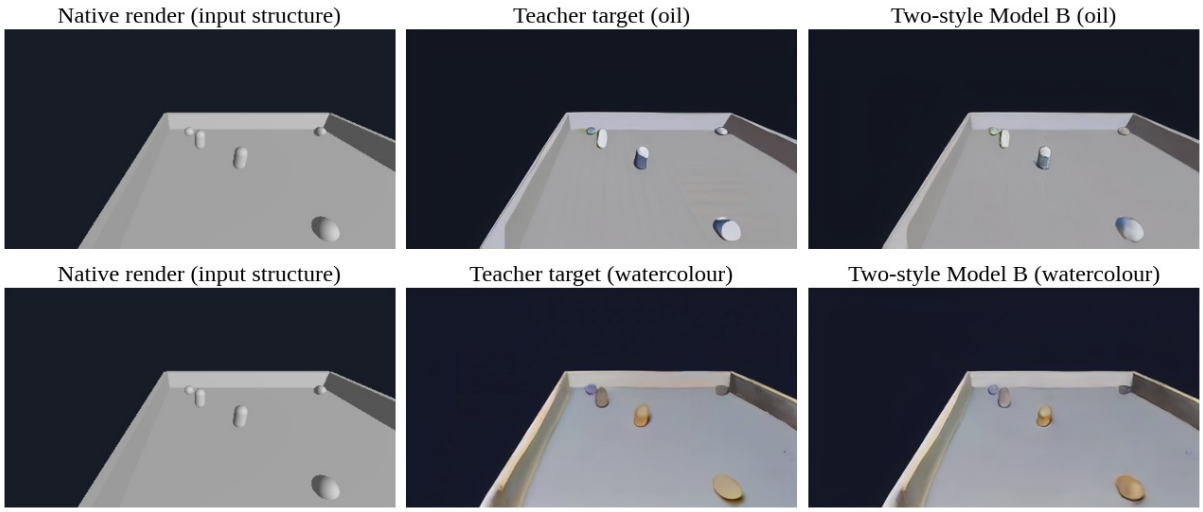


Figure 4.5: Native input, teacher target and model output for one sealed test frame

Figure 4.5 shows a sealed test frame (session f1f9c484, the stride-10 frame with the most entity pixels, chosen automatically) rendered through exactly the evaluated path with one frame of history. Geometry and object placement are those of the engine in every column; only appearance changes. The model reproduces each style's palette and the blue/ochre (oil) or cream/ochre (watercolour) tinting of the entities. It renders the floor's painted texture more softly than the teacher, consistent with its higher temporal stability. The styles are mild because the game is grey and plainly lit, and stronger teacher settings fail the alignment gates by inventing edges.

4.3.4 Value of Frame History

The first live version ran without frame history. Scoring the same model on the same test frames as a cold start every frame isolates what history contributes (Table 4.9; Figure 4.6).

Table 4.9: The same model with and without one frame of history (test frames)

Style	PSNR (dB)	LPIPS	Sharpness	Flicker
Oil: with → without	35.33 → 33.58	0.0225 → 0.0334	0.946 → 0.930	0.00566 → 0.00509
Watercolour: with → without	34.79 → 32.82	0.0243 → 0.0359	0.937 → 0.918	0.00614 → 0.00571

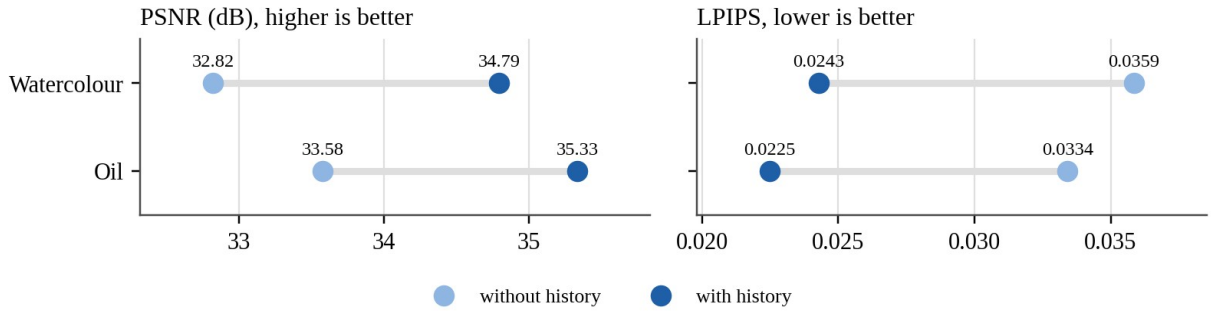


Figure 4.6: Image quality with and without frame history

History is worth about 1.8–2.0 dB and about one third of LPIPS, because the model was trained almost entirely with history and a cold start is off its training distribution. Flicker is slightly *lower* without history, consistent with the cold-start output being a little softer. This result motivated wiring the full temporal path into the live runtime.

4.3.5 Live Equivalence: the Game Shows the Evaluated Model

A live renderer can silently differ from the evaluated model. Each link from the evaluator to the screen was checked with identical inputs (Table 4.10).

Table 4.10: Equivalence chain from evaluator to live game

Link	Maximum difference	Evidence
Exported LiveStep graph vs Python evaluator (RGB / mask)	2.4×10^{-6} / 2.1×10^{-5}	unit tests; real test frames
TensorRT strict FP32 vs PyTorch (RGB / fed-back features)	1.6×10^{-6} / 7.3×10^{-6}	m10_live_temporal_engines.json
C++ motion vs engine GDScript motion, 120 lockstep frames	7.5×10^{-8} UV	m10_live_temporal_parity.log
Live game frame vs evaluated model	1/255 (rounding); 0 pixels above	m10_live_temporal_parity.json
Spawned enemy mid-game: no-history pixels, C++ vs engine ID map	998 = 998 pixels	m10_spawn_parity.log

The deployed FP16 engine was accepted against a tolerance fixed before it was measured: mean |ΔPSNR| < 0.05 dB and mean |pixel difference| < 1/255, over 99 real test pairs per style against the strict-FP32 engine. It passes with 0.044 dB (oil) and 0.039 dB (watercolour), and a mean pixel difference of 0.0004. The maximum single-pixel difference is 0.41 / 0.52 and the worst frame moves 0.74 dB. On inspection, every pixel off by more than 0.1 lies on a one-pixel object outline, where FP16 shifts an edge by a fraction of a pixel. FP16 therefore inherits the FP32 equivalence chain within this tolerance, not bit for bit.

4.3.6 Real-Time Performance and Robustness

Figure 4.7 and Table 4.11 report the live measurements on the playable game, with the GPU otherwise idle and the window on the real display. Figure 4.8 shows the per-frame time of the 3,000-frame stability run.

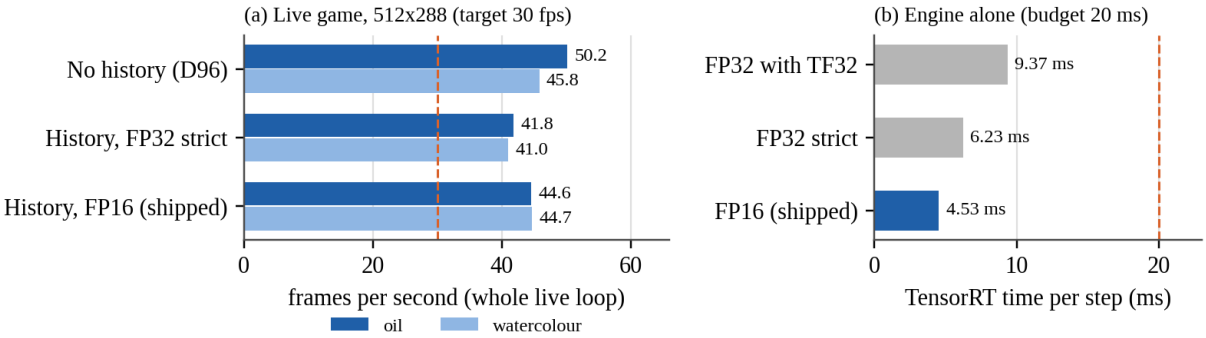


Figure 4.7: Live frame rate of the whole loop, and TensorRT latency alone

Table 4.11: M10 exit criteria, measured on the playable game

Criterion (KB target)	Result	Verdict
Stable live loop	3,000 frames, 0 fallbacks; median 22.4 ms (first half) vs 22.3 ms (second half); p95 24.5 ms, p99 25.2 ms	Met
About 30 fps at 512 × 288	44.6 fps oil, 44.7 fps watercolour (FP16); 41.8 / 41.0 fps strict FP32	Met
Standalone TensorRT ≤ about 20 ms	4.53 ms FP16 (p95 5.28 ms); 6.23 ms strict FP32; 9.37 ms with TF32	Met
Renderer failure does not break gameplay	Every 50th styled frame forced to fail: 8 of 8 fell back to the native frame, play continued (561 physics frames, 14.5 m travelled), next styled frame rendered; missing engine → native-only play with logged error	Met

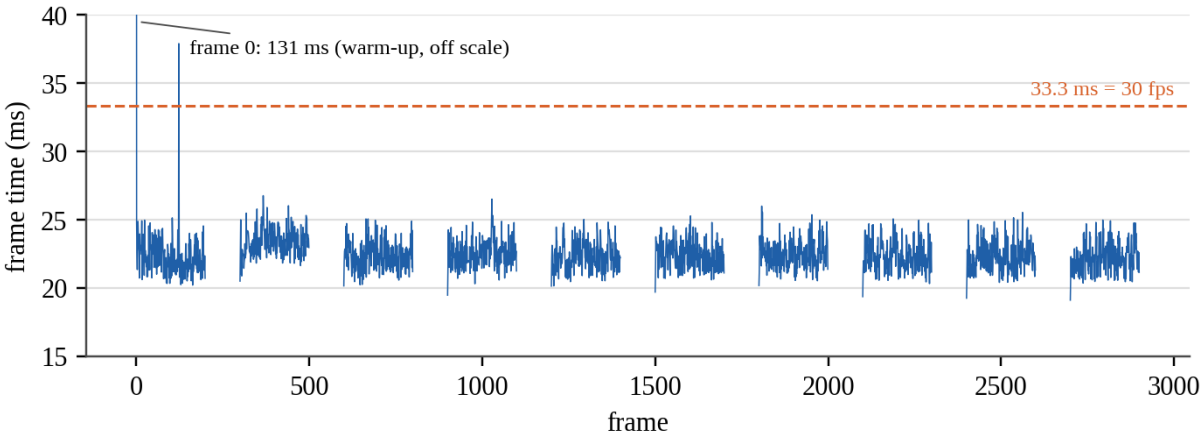


Figure 4.8: Per-frame time over the 3,000-frame stability run

Strict FP32 (6.2 ms) is faster than FP32 with TF32 enabled (9.4 ms) on this GPU, and is exact, so TF32 was not used. FP16 is 27 % faster again and passed its tolerance, so it ships. Only frame 0 exceeds 38 ms (131 ms, a one-off warm-up). The gaps in Figure 4.8 are the scripted native-mode intervals, in which the network does not run. Rendering the passes in parallel viewports, an optimisation planned earlier, was not needed.

Manual play session. On 23 October 2026 the game was also played by hand with logging enabled (`eval/logs/user_playtest.csv`). Over 2,059 styled frames (1,741 oil, 318 watercolour) and 16 mode switches, the median loop time was 24.9 ms (40.1 fps) for oil and 26.3 ms (38.0 fps) for watercolour, with **0 fallbacks**. This session was free play rather than lockstep, with ordinary desktop applications open, so it is slower than the idle-GPU measurements. It confirms the target is met under normal use.

4.3.7 Assessment of the Objectives

Table 4.12: Objectives and the evidence for each

Objective	Status	Evidence
O1 Structural data pipeline	Achieved	24/24 game sessions and 29/30 seed sessions validated; corrupted session rejected; recording and live capture byte-identical on test frames
O2 Offline supervision with gates	Achieved	Two gate-passing game styles; 11 of 23,760 frames excluded by an explicit rule
O3 Image quality (sharpness ≥ 0.90, travel ≥ 0.80)	Achieved	Sharpness 0.946 / 0.937; travel 0.995 / 0.987 (Tables 4.6, 4.8)
O4 Temporal stability beyond noise	Achieved	−15.2 % (B vs A) and −10.7 % (D vs C) against a 3.85 % threshold; 0.54× / 0.48× teacher flicker
O5 Controlled ablation	Achieved (as an evaluation)	All rungs at parity on one scoring path; result: U-Net best, transformer refuted at three data sizes
O6 Live style switching	Achieved	One model, two styles; outputs 4× closer to own style; switch keeps history
O7 Real-time deployment	Achieved	44.6 fps; 4.53 ms engine; 0 fallbacks in 3,000 frames; failure recovery (Table 4.11)
O8 EPAM improves output	Not achieved	F − F0 = −0.0127 sharpness, inside noise; lifecycle criteria met

4.3.8 Failure Cases and Limitations

- **One game, one scene family.** All results come from this project's arena scenes and two mild styles. They say nothing about other games or art styles.
- **The teacher's flicker is the ceiling.** The renderer halves the teacher's flicker but remains about 57× the engine's on the game.
- **Single training runs on the game.** Game-level seed variance was not measured, so small differences between game models are not claimed.
- **Gate-limited style strength.** Stronger teacher settings invent edges and fail the gates. On a 45-frame probe, a setting can pass and then fail the full session, which is why full-session gating is mandatory.
- **Resolution.** At 768 × 432 the teacher produces a different look with the same settings. A pre-registered probe found that the best re-tuned 768 settings were no closer to the committed look than the unstyled native frame (LPIPS 0.094 / 0.100 against 0.099 / 0.117), so the high-resolution dataset was stopped before the two-day generation run. The 768 × 432 pipeline itself works and is tested.
- **Free play is not lockstep.** In free play a physics tick can occasionally fall between capture passes; all equivalence checks were run in lockstep.

- **EPAM and structured entity use are untested where they should matter:** many entities, recycled IDs, and a temporally consistent teacher.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary of the Contributions

The project set out to add a learned artistic style to a playable game without letting a neural network decide anything about the game itself. It achieved this with a complete, verified pipeline: engine instrumentation; a validated dataset of 23,760 frames recorded from the real game; an offline diffusion teacher whose outputs are admitted only through four pre-registered alignment gates; a temporally conditioned, two-style U-Net; and an ONNX → TensorRT → C++ → Godot runtime.

The principal technical contributions are:

1. **An engine-grounded rendering design** in which the engine's structure, analytic motion and entity identity are used according to their meaning: structure as input, motion for warping, identity for masks and memory keys. None of them becomes a source of gameplay truth.
2. **Gated supervision from a diffusion teacher**, including a stylisation gate that rejects a teacher that does nothing, and a documented per-frame exclusion rule.
3. **A style-free encoder with FiLM decoding**, allowing one 8.39 M-parameter model to render two styles and to keep its temporal memory across a live style switch.
4. **A verified live deployment**: 44.6 fps in the playable game, 4.53 ms per FP16 network step, and a link-by-link proof that the live frame equals the evaluated model's frame to within rounding.
5. **Pre-registered, honestly reported evaluation**, including two negative results: the transformer renderer did not beat the U-Net at parameter parity, and persistent appearance memory did not help on this data.

Quantitatively, on held-out sessions the deployed model reaches 35.33 dB (oil) and 34.79 dB (watercolour) PSNR with sharpness 0.946 and 0.937. It reproduces 98.7–99.5 % of the teacher's palette shift and flickers about half as much as its teacher. Its practical relevance is a demonstration, on consumer hardware, that a game can be restyled live by a learned model while remaining exactly the same game. Its limitations are the narrow evidence base (one simple game, mild styles, 512 × 288) and a teacher whose own instability bounds temporal quality.

5.2 Future Work

The following directions follow from the measured limitations. They are proposals, not completed work.

- **A temporally consistent teacher.** Teacher flicker bounds both the renderer's stability and EPAM's usefulness. Options include video-consistent diffusion, authored per-entity textures, or post-processing the targets for temporal consistency.
- **Re-testing EPAM where it should matter:** scenes with many entities, occlusions and recycled IDs, together with a consistent teacher, as a new pre-registered experiment with repeated seeds.
- **Higher resolution with a matched look,** for example by generating targets at 512 × 288 and learning super-resolution, or by re-designing styles for 768 × 432 and accepting the new look as a new style.
- **Optional adversarial sharpening** (a patch discriminator) as supervision only, which the project's rules permit if non-adversarial training proves too smooth.
- **Broader games and styles,** including textured, well-lit scenes where stronger styles can pass the alignment gates, and human-play data to complement scripted routes.
- **Seed-variance measurement on the game,** so that small differences between game models can be tested for significance.
- **Further optimisation,** such as INT8 or FP8 inference after FP16, parallel capture passes, and evaluation on other GPUs.

REFERENCES

[1] A. Tewari et al., "State of the art on neural rendering," Comput. Graph. Forum, vol. 39, no. 2, pp. 701–727, 2020, doi: 10.1111/cgf.14022.

[2] T. Saito and T. Takahashi, "Comprehensible rendering of 3-D shapes," ACM SIGGRAPH Comput. Graph., vol. 24, no. 4, pp. 197–206, 1990, doi: 10.1145/97880.97901.

[3] S. W. Kim, Y. Zhou, J. Philion, A. Torralba, and S. Fidler, "Learning to simulate dynamic environments with GameGAN," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 1228–1237, doi: 10.1109/CVPR42600.2020.00131.

[4] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter, "Diffusion models are real-time game engines," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025, arXiv:2408.14837.

[5] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret, "Diffusion for world modeling: Visual details matter in Atari," in Advances in Neural Information Processing Systems (NeurIPS), 2024, arXiv:2405.12399.

[6] J. Bruce et al., "Genie: Generative interactive environments," arXiv preprint arXiv:2402.15391, 2024.

[7] Godot Engine contributors, "Godot Engine documentation." [Online]. Available: https://docs.godotengine.org/en/stable/ (accessed Oct. 23, 2026).

[8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10674–10685, doi: 10.1109/CVPR52688.2022.01042.

[9] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2023, pp. 3813–3824, doi: 10.1109/ICCV51070.2023.00355.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[11] ONNX Project, "Open Neural Network Exchange (ONNX)." [Online]. Available: https://onnx.ai/ (accessed Oct. 23, 2026).

[12] NVIDIA Corporation, "NVIDIA TensorRT documentation." [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/ (accessed Oct. 23, 2026).

[13] StructRender project, "StructRender: System architecture and knowledge base, including the Decision Log (Decisions 1–102)," unpublished technical document, docs/StructRender_KB.md, Oct. 2026.

[14] S. R. Richter, H. A. Alhaija, and V. Koltun, "Enhancing photorealism enhancement," IEEE Trans. Pattern Anal. Mach. Intell., vol. 45, no. 2, pp. 1700–1715, 2023, doi: 10.1109/TPAMI.2022.3166687.

[15] M. Ruder, A. Dosovitskiy, and T. Brox, "Artistic style transfer for videos," in Pattern Recognition (GCPR 2016, Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2016, pp. 26–36, doi: 10.1007/978-3-319-45886-1_3.

[16] W.-S. Lai, J.-B. Huang, O. Wang, E. Shechtman, E. Yumer, and M.-H. Yang, "Learning blind video temporal consistency," in Computer Vision – ECCV 2018 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2018, pp. 179–195, doi: 10.1007/978-3-030-01267-0_11.

[17] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Advances in Neural Information Processing Systems (NeurIPS), 2020, arXiv:2006.11239.

[18] Hugging Face model repository "stable-diffusion-v1-5/stable-diffusion-v1-5," revision 451f4fe1. [Online]. Available: https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5 (accessed Oct. 23, 2026).

[19] Hugging Face model repository "lllyasviel/sd-controlnet-depth," revision 35e42a3e. [Online]. Available: https://huggingface.co/lllyasviel/sd-controlnet-depth (accessed Oct. 23, 2026).

[20] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 5967–5976, doi: 10.1109/CVPR.2017.632.

[21] Y. Wu and K. He, "Group normalization," in Computer Vision – ECCV 2018 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2018, pp. 3–19, doi: 10.1007/978-3-030-01261-8_1.

[22] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 2414–2423, doi: 10.1109/CVPR.2016.265.

[23] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in Computer Vision – ECCV 2016 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2016, pp. 694–711, doi: 10.1007/978-3-319-46475-6_43.

[24] V. Dumoulin, J. Shlens, and M. Kudlur, "A learned representation for artistic style," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017, arXiv:1610.07629.

[25] X. Huang and S. Belongie, "Arbitrary style transfer in real-time with adaptive instance normalization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 1510–1519, doi: 10.1109/ICCV.2017.167.

[26] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in Proc. AAAI Conf. Artif. Intell., vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.11671.

[27] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, "SDEdit: Guided image synthesis and editing with stochastic differential equations," in Proc. Int. Conf. Learn. Represent. (ICLR), 2022, arXiv:2108.01073.

[28] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004, doi: 10.1109/TIP.2003.819861.

[29] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," Commun. ACM, vol. 60, no. 6, pp. 84–90, 2017, doi: 10.1145/3065386.

[30] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 586–595, doi: 10.1109/CVPR.2018.00068.

[31] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 6228–6237, doi: 10.1109/CVPR.2018.00652.

[32] C. R. A. Chaitanya, A. S. Kaplanyan, C. Schied, M. Salvi, A. Lefohn, D. Nowrouzezahrai, and T. Aila, "Interactive reconstruction of Monte Carlo image sequences using a recurrent denoising autoencoder," ACM Trans. Graph., vol. 36, no. 4, pp. 1–12, 2017, doi: 10.1145/3072959.3073601.

[33] L. Xiao, S. Nouri, M. Chapman, A. Fix, D. Lanman, and A. Kaplanyan, "Neural supersampling for real-time rendering," ACM Trans. Graph., vol. 39, no. 4, 2020, doi: 10.1145/3386569.3392376.

[34] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 9992–10002, doi: 10.1109/ICCV48922.2021.00986.

[35] O. Nalbach, E. Arabadzhiyska, D. Mehta, H.-P. Seidel, and T. Ritschel, "Deep shading: Convolutional neural networks for screen space shading," Comput. Graph. Forum, vol. 36, no. 4, pp. 65–78, 2017, doi: 10.1111/cgf.13225.

[36] T. Park, M.-Y. Liu, T.-C. Wang, and J.-Y. Zhu, "Semantic image synthesis with spatially-adaptive normalization," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 2332–2341, doi: 10.1109/CVPR.2019.00244.

[37] T.-C. Wang, M.-Y. Liu, J.-Y. Zhu, G. Liu, A. Tao, J. Kautz, and B. Catanzaro, "Video-to-video synthesis," in Advances in Neural Information Processing Systems (NeurIPS), 2018, arXiv:1808.06601.

[38] A. Mallya, T.-C. Wang, K. Sapra, and M.-Y. Liu, "World-consistent video-to-video synthesis," in Computer Vision – ECCV 2020 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2020, pp. 359–378, doi: 10.1007/978-3-030-58598-3_22.

[39] Godot Engine contributors, "godot-cpp: C++ bindings for the Godot script API (GDExtension)," GitHub repository. [Online]. Available: https://github.com/godotengine/godot-cpp (accessed Oct. 23, 2026).

[40] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, arXiv:1912.01703.

[41] Hugging Face, "Diffusers: State-of-the-art diffusion models," GitHub repository. [Online]. Available: https://github.com/huggingface/diffusers (accessed Oct. 23, 2026).

[42] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, arXiv:1711.05101.

[43] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017, arXiv:1608.03983.

APPENDICES

Appendix A: Deployed Model Configuration (abridged)

```
{ "config_version": "game-b-two-style-v1",
  "data":  { "styles": ["oil_impasto", "watercolor_bold_s055"],
             "split_file": "configs/data/game_split_v1.json",
             "frame_exclusions":
                 "configs/data/game_frame_exclusions_v1.json" },
  "model": { "in_channels": 7, "out_channels": 3, "base_channels": 32,
             "image_size": [288, 512], "arch": "style_temporal_unet" },
  "train": { "seed": 20260905, "epochs": 60, "lr": 0.0002,
             "weight_decay": 0.0001,
             "amp_dtype": "bfloat16", "batch_size": 4, "accum_steps": 2,
             "eval_every_epochs": 2,
             "early_stopping": { "patience_evals": 6,
                                 "min_rel_delta": 0.001 } },
  "loss":  { "l1": 1.0, "perceptual": 0.5, "perceptual_net": "alex",
             "temporal": 0.5 } }
```

Appendix B: Milestone Scorecard

Table B.1: Milestones M0–M10 and their verdicts

Milestone	Verdict
M0 Repository and minimal Godot game	Complete
M1 Structural export (depth, normals, albedo, motion, entity ID)	Complete
M2 Dataset recorder and validator	Complete
M3 Offline teacher targets and four alignment gates	Complete
M4 CNN / U-Net baseline	Passes at every dataset size
M5 StructFormer baseline	Fails: below the U-Net at 644, 10,890 and 20,790 frames
M6 Temporal reprojection	Passes: −11.7 % flicker; replicated −10.7 % on new data
M7 Structured entity awareness	Complete: all three criteria asserted by tests
M8 EPAM	Criteria 1–3 met; criterion 4 (improves output) not met
M9 Full evaluation (ablation ladder)	Complete
M10 ONNX → TensorRT → C++ → Godot	Passes all exit criteria

A failed milestone means that a pre-registered criterion was not met, not that work is missing; such failures are findings about this data.

Appendix C: Reproduction Commands and Evidence Files

```
# Python environment (conda env 'dl'); automated tests
python -m pytest -q

# Score the deployed model on the sealed test sessions (oil shown)
python -m scripts.eval_game checkpoints/game_b_two_style/best.pt \
    configs/data/game_split_v1.json oil_impasto \
    eval/reports/game_b_two_style_test_oil.json \
```

```
    configs/data/game_frame_exclusions_v1.json

# Build the live engines on this GPU, then play
# (keys: 1 oil, 2 watercolour, 0 native)
python -m scripts.m10_live_temporal_engines
python -m scripts.m10_fp16_live
E=$PWD/artifacts/engines
SR_ENGINE_OIL=$E/game_b2_live_oil_impasto_fp16.engine \
SR_ENGINE_WATERCOLOR=$E/game_b2_live_watercolor_bold_s055_fp16.engine \
  godot --path game_project -s res://tests/runtime/live_game.gd

# Figures of this report
PYTHONPATH=$PWD python docs/lab_report/build/make_figures.py
```

Table C.1: Evidence files behind the reported numbers

Claim	File(s)
Ablation ladder; temporal error	eval/reports/m9_ladder.json; eval/reports/m9_temporal_error.json
StructFormer scaling	eval/reports/ladder_v3_test.json
EPAM F vs F0	eval/reports/m8_f_vs_f0.json
Deployed model training	eval/reports/game_b_two_style_train.json; eval/logs/game_b_two_style/
Game test results	eval/reports/game_b_two_style_test_oil.json; …_test_watercolor.json; game_b_oil_test_d94frames.json
Without history	eval/reports/game_b_two_style_test_*_nohistory.json
Style acquisition	eval/reports/game_style_acquisition.json
Frame exclusions	configs/data/game_frame_exclusions_v1.json
Live equivalence	eval/reports/m10_live_temporal_engines.json; m10_live_temporal_parity.json; eval/logs/m10_spawn_parity.log
FP16 tolerance	eval/reports/m10_fp16_live.json
Live speed, stability, failures	eval/logs/m10_final/
Manual play session	eval/logs/user_playtest.csv
High-resolution probe	eval/reports/hr_style_match_probe.json
Decisions cited as "KB"	docs/StructRender_KB.md (Decision Log)

Appendix D: Frames Excluded by the Alignment Gates

Table D.1: The 11 excluded frames (game dataset)

Session	Frame(s)	Style and failing gate	Split
ds_1790534482_6d509a33	119	watercolour: entity silhouette	train
ds_1790534719_716f96e0	270, 285	watercolour: spurious edges	train
ds_1790534971_62b8325b	110	watercolour: spurious edges	train
ds_1790535959_640a0d22	30	oil: entity silhouette	train
ds_1790536212_9fbff551	153	watercolour: spurious edges	train
ds_1790537000_85c67b3e	846	watercolour: boundary F	train
ds_1790537747_81203cd9	879, 901	watercolour: boundary F	validation
ds_1790538263_4815ad01	86, 845	watercolour: spurious edges	test